WEEK 1 • COMBINED FOUNDATIONS

Introduction to Agentic AI
& Research Methodology

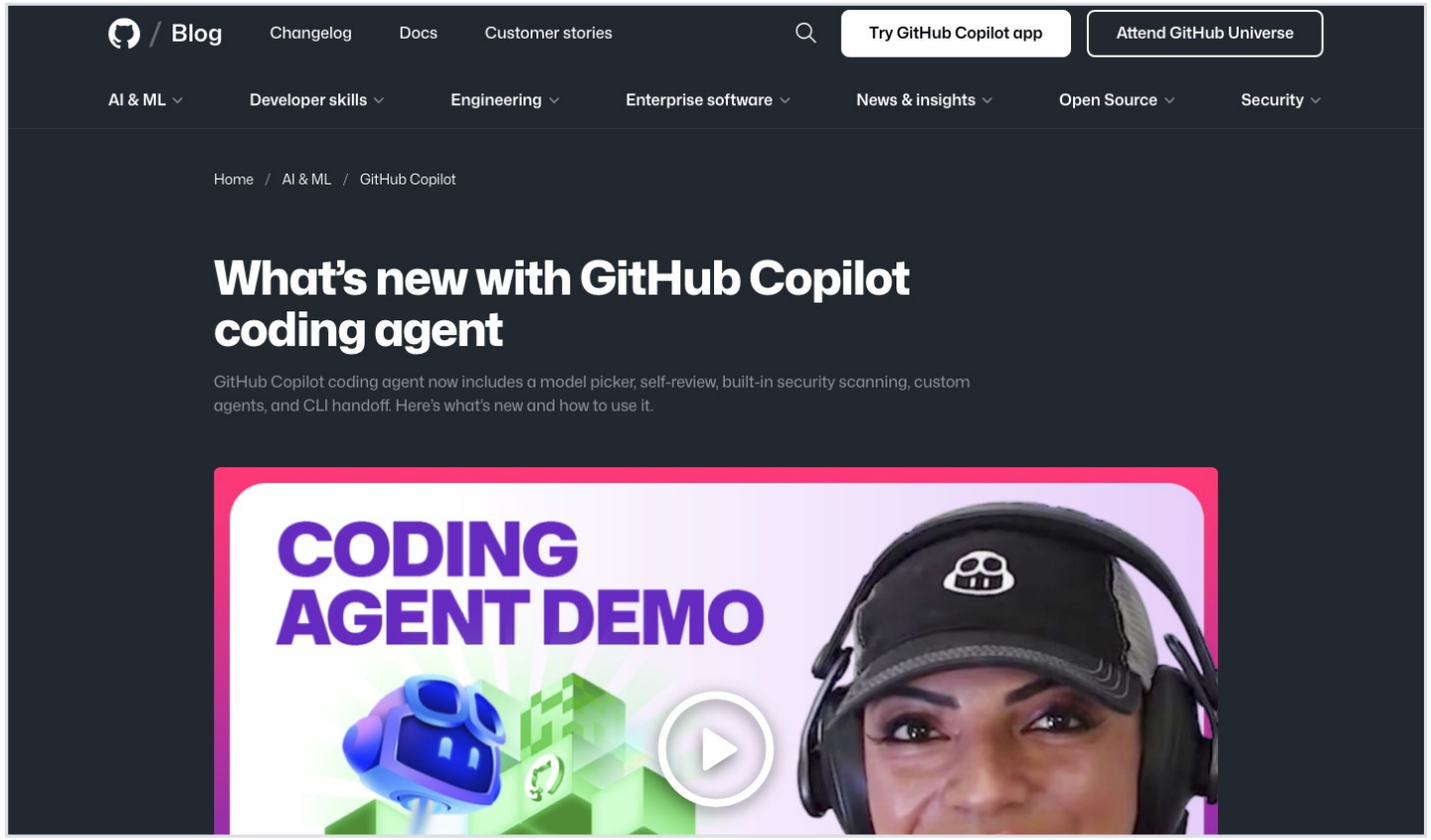
Reasoning & Planning • CoT → ReAct → Reflexion → ToT/GoT

CO5151 - Advanced Agentic Artificial Intelligence
School of Computer Science and Engineering
Lê Xuân Bách • 9 September 2026

Flipped format: pre-read Lecture Notes Weeks 1-2; class time is for discussion, demo, critique, and decisions.

News snapshot 1/3

Read the headline as a research claim: what changed, and what evidence would make it credible?



26 FEBRUARY 2026 — **GitHub Blog**

Coding agents add self-review, security scans, and handoff

Why it matters

A coding agent is no longer only a chat surface: it works asynchronously, changes repository state, runs checks, and returns an artifact for review.

Question

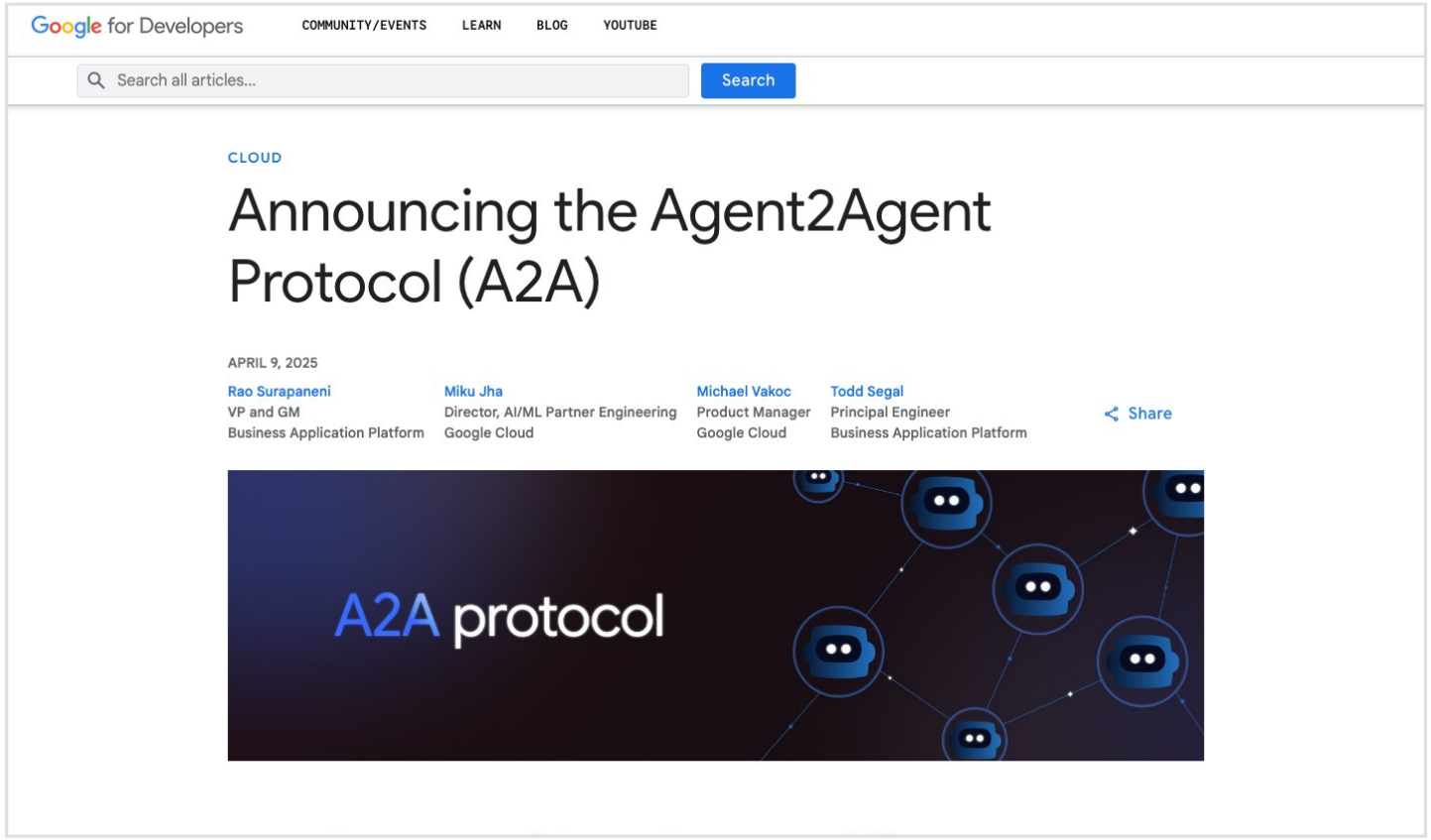
Which loop elements are visible-and where is the human checkpoint?

Open the article ↗

Snapshot captured 9 Sep 2026 • Source: GitHub Blog • https://github.blog/ai-and-ml/github-copilot/whats-new-with-github-copilot-coding-agent/

News snapshot 2/3

Read the headline as a research claim: what changed, and what evidence would make it credible?



9 APRIL 2025 — **Google Developers Blog**

Agent2Agent turns collaboration into a protocol problem

Why it matters

A2A standardizes capability discovery, task state, messages, and artifacts so agents from different vendors or frameworks can coordinate.

Question

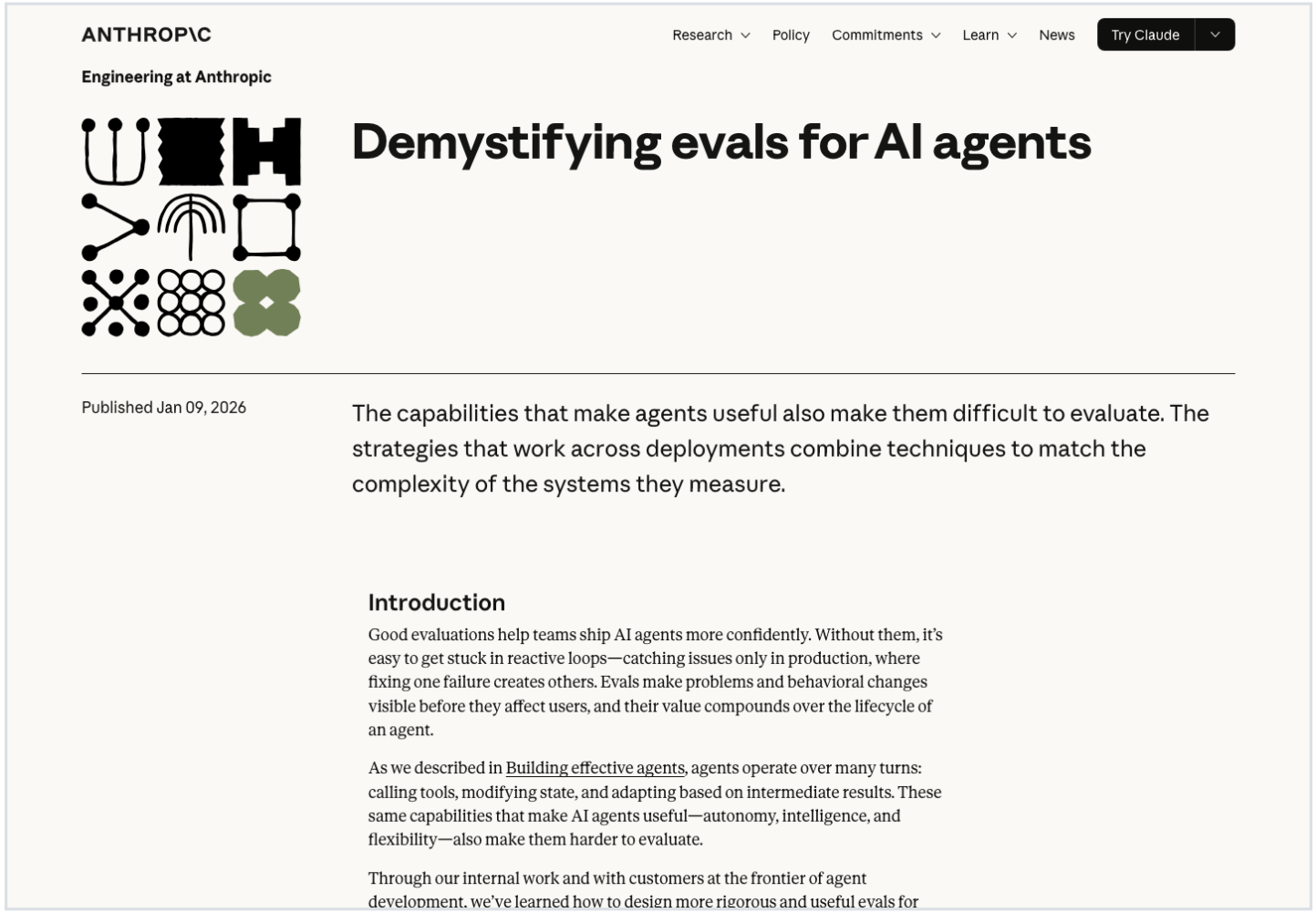
Does communication create a multi-agent system-or only connect tools?

Open the article ↗

Snapshot captured 9 Sep 2026 • Source: Google Developers Blog • https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/

News snapshot 3/3

Read the headline as a research claim: what changed, and what evidence would make it credible?



ANTHROP\C

Research Policy Commitments Learn News Try Claude

Engineering at Anthropic

Demystifying evals for AI agents

Published Jan 09, 2026

The capabilities that make agents useful also make them difficult to evaluate. The strategies that work across deployments combine techniques to match the complexity of the systems they measure.

Introduction

Good evaluations help teams ship AI agents more confidently. Without them, it's easy to get stuck in reactive loops—catching issues only in production, where fixing one failure creates others. Evals make problems and behavioral changes visible before they affect users, and their value compounds over the lifecycle of an agent.

As we described in Building effective agents, agents operate over many turns: calling tools, modifying state, and adapting based on intermediate results. These same capabilities that make AI agents useful—autonomy, intelligence, and flexibility—also make them harder to evaluate.

Through our internal work and with customers at the frontier of agent development, we've learned how to design more rigorous and useful evals for

9 JANUARY 2026 — **Anthropic Engineering**

The capabilities that make agents useful also make them hard to evaluate

Why it matters

Multi-turn behavior changes state and adapts to intermediate results, so one-shot accuracy is insufficient evidence of reliability.

Question

What must an evaluation capture besides the final answer?

Open the article ↗

Snapshot captured 9 Sep 2026 • Source: Anthropic Engineering • https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

START THE LECTURE HERE

Opening question: what makes all three agentic?

Products expose the loop; protocols connect loops; evaluation tests the whole system

PRODUCT

Coding agent

acts on repository state

PROTOCOL

A2A

coordinates independent agents

EVIDENCE

Agent evals

measure trajectories, not only answers

What makes the loop agentic - and what evidence would make its claims credible?

▶ Video primer: AI Agents, Clearly Explained

Opening sources: GitHub (2026) • Google A2A (2025) • Anthropic agent evals (2026)

NEXT HARD DEADLINE

SUN 13 SEP 2026

23:59 ICT

D1 - PROJECT PROPOSAL

Submit the 3-4 page proposal, project idea one-pager, and prepare the 5-minute pitch.

COMPLETE IN CLASS

BY 20:29

Form or swap groups and rank seminar topics

Finish group swaps and topic ranking before leaving this room.

Learning objectives

By the end of this session, you can…

1

Define and classify

Distinguish a chatbot, workflow, script, and autonomous agent; place systems on an autonomy spectrum.

2

Decompose an agent

Map perception/context, reasoning, memory, and action/tools onto one control loop.

3

Compare reasoning methods

Explain CoT, self-consistency, ReAct, Reflexion, ToT/GoT, and their cost/feedback assumptions.

4

Read and critique papers

Use three passes, five presentation parts, and assumption-level critique rather than summary.

Sources: Week 1 and Week 2 lecture notes • PLAN.md §4, Week 1

Course contract

Research seminar + group project • no midterm and no final exam

25%

Seminar 1

Core agent methods

25%

Seminar 2

Advanced topics

50%

Group project

Build, evaluate, defend

✓ What earns marks

Mechanisms, critical analysis, defensible experiments, quantitative evidence, and clear Q&A.

! What students must do

Attend seminar critique, declare AI use, respect deadlines, and be able to defend every submitted claim.

Course motto: learning through working.

Source: PLAN.md §§1-2 and §7

SESSION ROADMAP

Today’s 149-minute map

A flipped session: concept checks, live traces, critique, and organization

1	2	3	4	5	6	7
00-08	08-15	15-35	35-70	70-80	80-115	115-149
News hook	**Contract**	**Agent foundations**	**Reasoning**	**Break**	**Research method**	**Organization**
3 snapshots	deadlines + grading	definition + anatomy	guided comparison	reset	live ReAct critique	groups + next actions

Keep one question visible all session: Where does the gain come from-and what does it cost?

Source: PLAN.md §4, Week 1 timeline

THINK • PAIR • SHARE

Opening question: is it really an agent?

Classify first; justify using observable behavior, not marketing language

L0

Autocomplete

One prompt → one completion

L1

Weather assistant

Chooses one function call → answers

L2

Extract → summarize → translate

Fixed pipeline authored by a developer

?

Research assistant

Chooses searches, revises, and decides when to stop

What evidence would move the final system up-or down-a level?

01

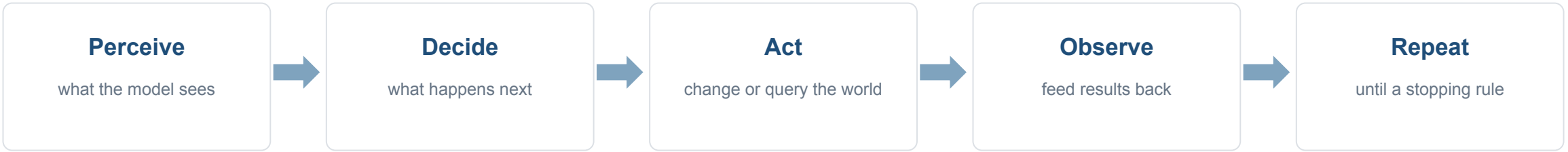
What counts as an agent?

A working definition, an autonomy spectrum, and a field map for critique.

A working definition

The loop-not a capability label-is the defining property

> **“An agent is a system in which a language model’s output determines the next action taken in an environment, and that action’s result is fed back into the model’s context before the next decision.”**



A single prompt-response call has no feedback loop. A two-step tool loop is minimal-but real-agency.

Source: Week 1 notes §1.1 • Course website §01

Boundary test: who chooses the path?

The key threshold is model control over steps and stopping

L0-L1

Call

No iterative decision loop

Autocomplete; one weather function call

L2

Workflow

Path authored at design time

extract → summarize → translate

L3+

Agent

Path chosen at run time

ReAct chooses tools and decides when to stop

Diagnostic question: is the path itself generated-or only the content inside a fixed path?

Source: Week 1 notes §1.2 • Anthropic, Building Effective Agents (2024)

An autonomy spectrum

Six levels-from a single call to continuous operation

autonomy increases →

Research use Check claimed vs. actual level.

Project use Name the level and match the evaluation bar.

Source: Week 1 notes §1.2 • Course website autonomy staircase

ARCHITECTURE DECISION

Workflow or agent? Choose deliberately

Predictability and flexibility trade off against latency, cost, and control

Workflow (L2)

Use when the task is well-defined and the path is stable.

✓ predictable sequence
✓ easier testing and permissions
✓ consistent latency

Risk: brittle when cases diverge from the authored path.

Agent (L3+)

Use when the path cannot be fully specified in advance.

✓ model selects steps and stop
✓ adapts to observations
✓ handles open-ended tasks

Risk: more latency, variability, and safety surface.

Design rule: start with the simplest architecture that satisfies the task evidence.

Source: Anthropic, Building Effective Agents (2024), as summarized in Week 1 notes

2022-2026: three phases

A lens for locating each paper's contribution-not a claim that research moved in lockstep

2022-23

PROMPTING

Elicit behavior from frozen models

CoT • ReAct • Reflexion • ToT

2024

INFRASTRUCTURE

Make agents buildable and measurable

MCP • SWE-bench • GAIA • AgentDojo

2025-26

TRAINING + ARCHITECTURE

Bake in reasoning; constrain capability

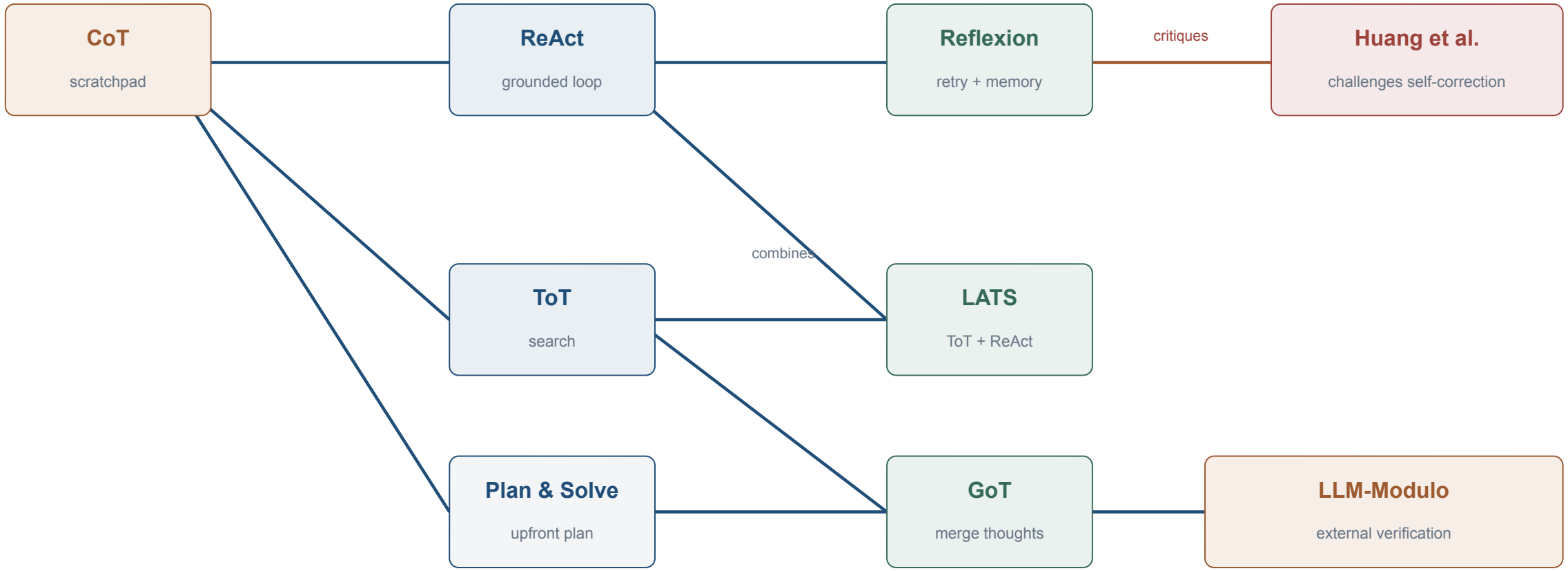
reasoning RL • computer use • A2A • CaMeL

Question the narrative: later phases can subsume earlier techniques-but may also coexist with them.

Source: Week 1 notes §1.3 • Course website §02

Ideas form a dependency graph

Reasoning and planning papers build on-and challenge-one another



Source: Course website §02 dependency graph • Week 2 lecture notes

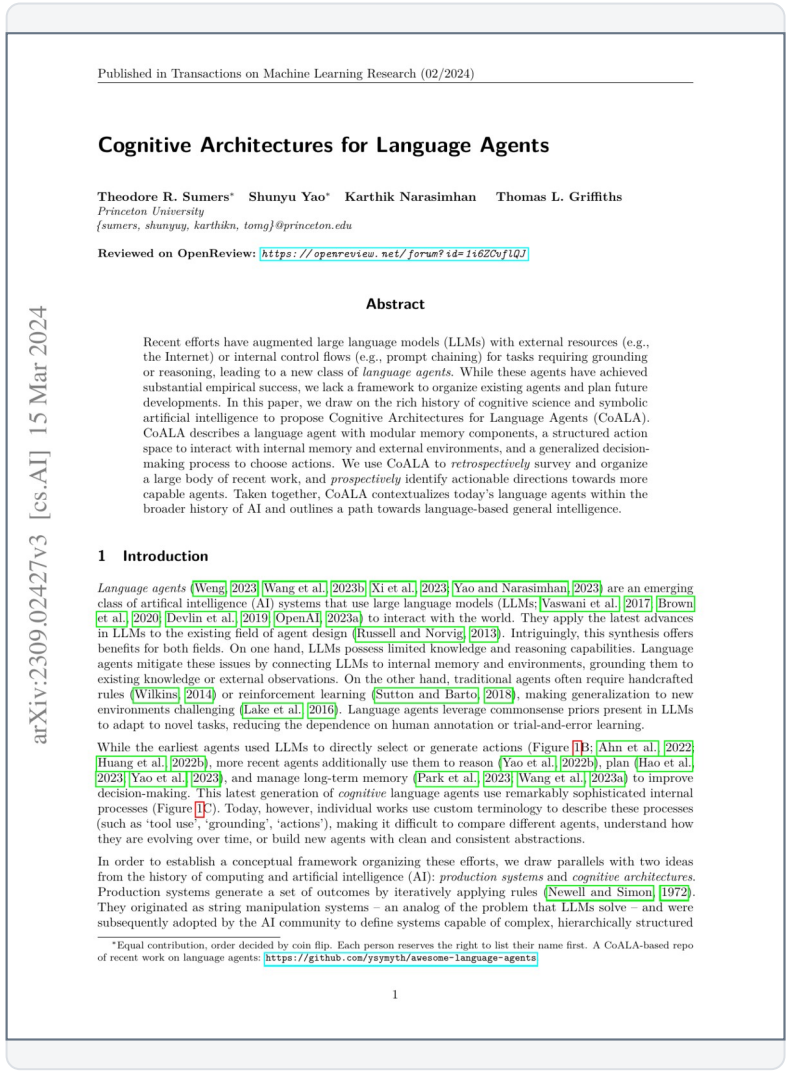
02

Anatomy of an agent

Four functional components wired by one control loop.

Paper spotlight: CoALA

Cognitive Architectures for Language Agents • TMLR 2024



Published in Transactions on Machine Learning Research (02/2024)

Cognitive Architectures for Language Agents

Theodore R. Sumers* **Shunyu Yao*** **Karthik Narasimhan** **Thomas L. Griffiths**
Princeton University
{sumers, shunyuy, karthikn, tomg}@princeton.edu

Reviewed on OpenReview: *https://openreview.net/forum?id=1i62CoflQJ*

Abstract

Recent efforts have augmented large language models (LLMs) with external resources (e.g., the Internet) or internal control flows (e.g., prompt chaining) for tasks requiring grounding or reasoning, leading to a new class of *language agents*. While these agents have achieved substantial empirical success, we lack a framework to organize existing agents and plan future developments. In this paper, we draw on the rich history of cognitive science and symbolic artificial intelligence to propose Cognitive Architectures for Language Agents (CoALA). CoALA describes a language agent with modular memory components, a structured action space to interact with internal memory and external environments, and a generalized decision-making process to choose actions. We use CoALA to *retrospectively* survey and organize a large body of recent work, and *prospectively* identify actionable directions towards more capable agents. Taken together, CoALA contextualizes today's language agents within the broader history of AI and outlines a path towards language-based general intelligence.

1 Introduction

Language agents (Weng, 2023; Wang et al., 2023b; Xi et al., 2023; Yao and Narasimhan, 2023) are an emerging class of artifical intelligence (AI) systems that use large language models (LLMs; Vaswani et al., 2017; Brown et al., 2020; Devlin et al., 2019; OpenAI, 2023a) to interact with the world. They apply the latest advances in LLMs to the existing field of agent design (Russell and Norvig, 2013). Intriguingly, this synthesis offers benefits for both fields. On one hand, LLMs possess limited knowledge and reasoning capabilities. Language agents mitigate these issues by connecting LLMs to internal memory and environments, grounding them to existing knowledge or external observations. On the other hand, traditional agents often require handcrafted rules (Wilkins, 2014) or reinforcement learning (Sutton and Barto, 2018), making generalization to new environments challenging (Lake et al., 2016). Language agents leverage commonsense priors present in LLMs to adapt to novel tasks, reducing the dependence on human annotation or trial-and-error learning.

While the earliest agents used LLMs to directly select or generate actions (Figure 1B; Ahn et al., 2022; Huang et al., 2022b), more recent agents additionally use them to reason (Yao et al., 2022b), plan (Hao et al., 2023; Yao et al., 2023), and manage long-term memory (Park et al., 2023; Wang et al., 2023a) to improve decision-making. This latest generation of *cognitive* language agents use remarkably sophisticated internal processes (Figure 1C). Today, however, individual works use custom terminology to describe these processes (such as 'tool use', 'grounding', 'actions'), making it difficult to compare different agents, understand how they are evolving over time, or build new agents with clean and consistent abstractions.

In order to establish a conceptual framework organizing these efforts, we draw parallels with two ideas from the history of computing and artificial intelligence (AI): *production systems* and *cognitive architectures*. Production systems generate a set of outcomes by iteratively applying rules (Newell and Simon, 1972). They originated as string manipulation systems – an analog of the problem that LLMs solve – and were subsequently adopted by the AI community to define systems capable of complex, hierarchically structured

*Equal contribution, order decided by coin flip. Each person reserves the right to list their name first. A CoALA-based repo of recent work on language agents: https://github.com/ysymyth/awesome-language-agents

1

Main contribution

Introduces a systematic architecture for language agents with modular memory, structured internal/external actions, and a generalized decision procedure.

Intuition

Treat an agent as a cognitive system whose memory and actions are organized around a decision loop, rather than as an LLM with an arbitrary list of add-ons.

Significance

Gives the course a common vocabulary for comparing systems, locating a paper's real contribution, and seeing which components it leaves assumed or unexplored.

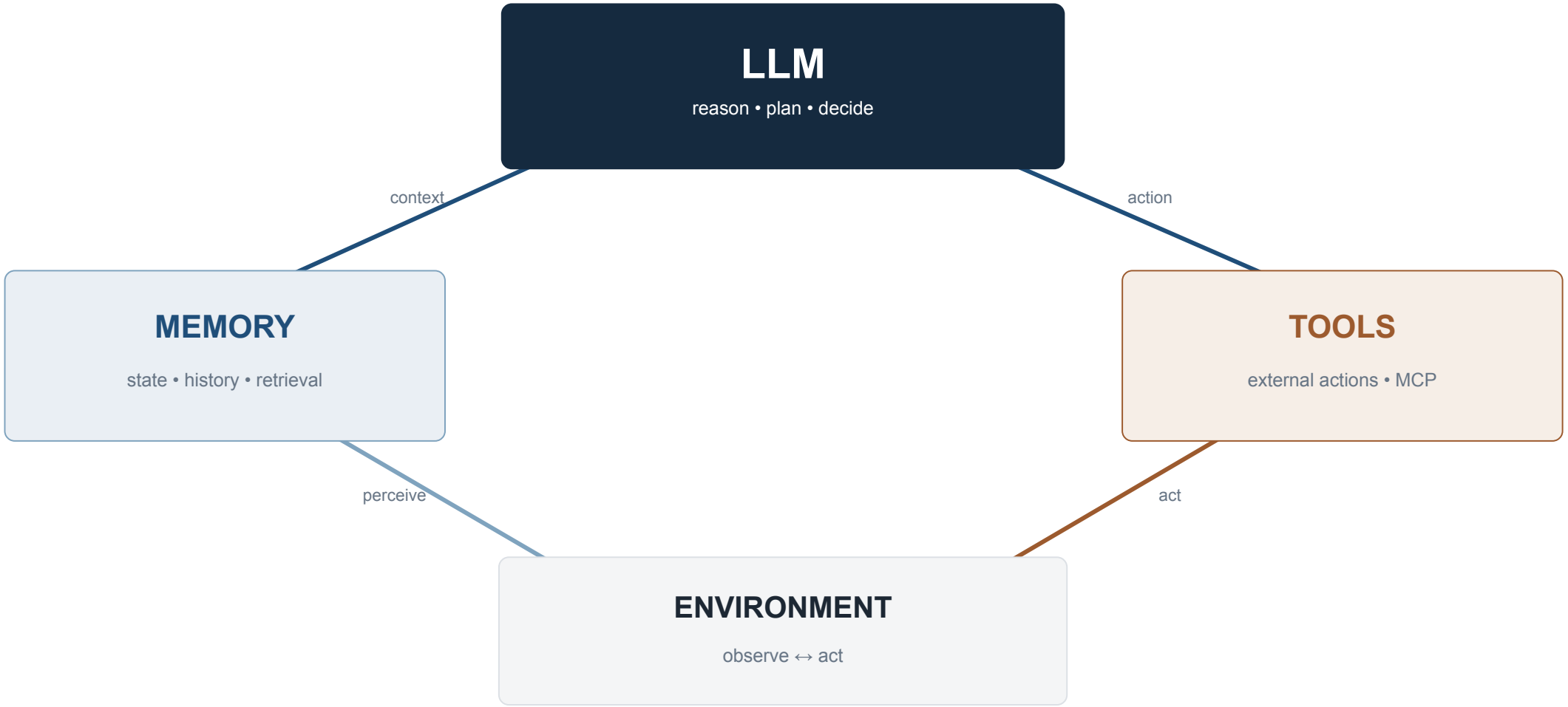
Open paper ↗

First page shown at left

Primary source: https://arxiv.org/abs/2309.02427

Four components, one loop

A compressed view of CoALA's cognitive architecture



Source: Sumers et al., CoALA (TMLR 2024) • Week 1 notes §2.1 • Course website §03

The control loop, made concrete

Most agent systems are variants of this skeleton

```
def agent_loop(task, tools, memory, max_steps):
    context = build_initial_context(task, memory)
    for step in range(max_steps):
        decision = llm(context)          # REASONING
        if decision.is_final_answer:
            return decision.answer
        result = execute(decision.action, tools)
                                         # ACTION
        memory.maybe_write(step, decision, result)
                                         # MEMORY
        context = update_context(context, decision, result)
                                         # PERCEPTION
    return fail("max steps exceeded")
```

ReAct

Split decision into Thought + Action.

Reflexion

Retry; write reflection into episodic memory.

ToT / LATS

Search over several decisions or trajectories.

One loop or several? What changes inside it?

Source: Week 1 notes §2.2 • Course website agent-loop pseudocode

Memory is a policy-not a larger context window

Choose what to save and what to retrieve

Coding assistant

→

Working memory

Project conventions during a long refactor

Support agent

→

Episodic memory

Relevant past tickets across sessions

Research assistant

→

Semantic memory

Facts already verified and reusable

Write policy

What gets saved-and when?

Retrieval policy

What returns to context-and when?

Source: Course website §03, “Memory, made concrete” • Week 1 notes §2.1

Where the semester fits

Use the anatomy to name a paper's actual scope

COMPONENT / PROPERTY	SEMINAR 1	SEMINAR 2
Reasoning	S1-1 Planning • S1-6 Reflection	S2-6 RL for Agents
Perception / context	S1-4 Retrieval-Augmented Agents	S2-7 Computer-use / Embodied
Memory	S1-3 Agent Memory	S2-4 Memory poisoning / isolation
Action / tools	S1-2 MCP • S1-5 Coding Agents	S2-3/4 Tools as attack surface
Multiple loops	S1-7 Orchestration	S2-1/2 Multi-agent systems
Evaluate whole loop	S1-8 Evaluation	S2-5 Safety • S2-8 Governance

Useful critique opener: "This paper contributes to ___ and assumes ___ is solved."

Source: Week 1 notes §2.3 • Course website component-to-seminar map

LIVE ACTIVITY

Watch the loop happen

The course site compares three scripted, zero-cost walkthroughs of the same task

Open: CO5151-Website/dist/week-01.html → Agent loop simulator

Track step count • LLM calls • tool calls • memory writes

Source: Course website §03b • assets/agent-sim.js

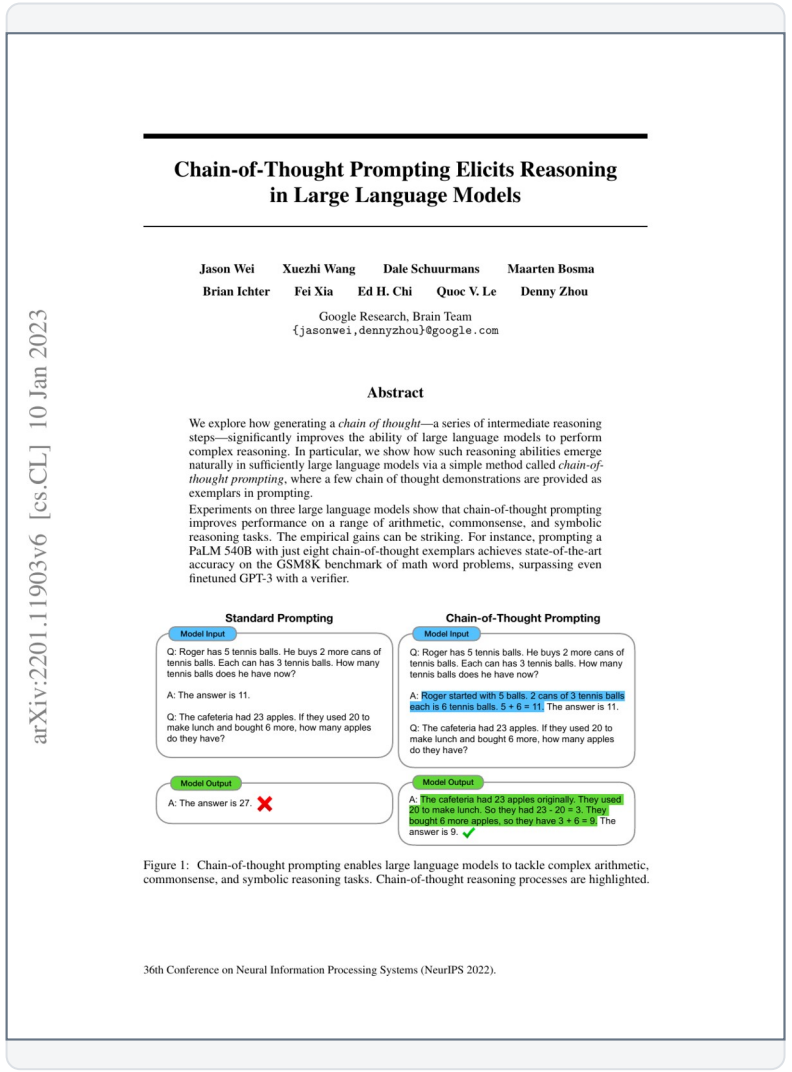
03

Reasoning & planning

Compare mechanisms on feedback quality, search structure, and test-time cost.

Paper spotlight: Chain-of-Thought

Chain-of-Thought Prompting Elicits Reasoning in Large Language Models • NeurIPS 2022



Chain-of-Thought Prompting Elicits Reasoning in Large Language Models

Jason Wei Xuezhi Wang Dale Schuurmans Maarten Bosma
Brian Ichter Fei Xia Ed H. Chi Quoc V. Le Denny Zhou
Google Research, Brain Team
{jasonwei,dennyzhou}@google.com

Abstract

We explore how generating a *chain of thought*—a series of intermediate reasoning steps—significantly improves the ability of large language models to perform complex reasoning. In particular, we show how such reasoning abilities emerge naturally in sufficiently large language models via a simple method called *chain-of-thought prompting*, where a few chain of thought demonstrations are provided as exemplars in prompting.

Experiments on three large language models show that chain-of-thought prompting improves performance on a range of arithmetic, commonsense, and symbolic reasoning tasks. The empirical gains can be striking. For instance, prompting a PaLM 540B with just eight chain-of-thought exemplars achieves state-of-the-art accuracy on the GSM8K benchmark of math word problems, surpassing even finetuned GPT-3 with a verifier.

Figure 1: Chain-of-thought prompting enables large language models to tackle complex arithmetic, commonsense, and symbolic reasoning tasks. Chain-of-thought reasoning processes are highlighted.

36th Conference on Neural Information Processing Systems (NeurIPS 2022).

Main contribution

Shows that few-shot demonstrations containing intermediate reasoning steps can substantially improve large-model performance on arithmetic, symbolic, and commonsense tasks.

Intuition

Generated intermediate tokens become a scratchpad: each partial step enters the context and turns one difficult prediction into a sequence of smaller conditioned predictions.

Significance

Established inference-time reasoning as a practical research direction and became the foundation for self-consistency, ReAct, search-based reasoning, and later reasoning-trained models.

Open paper ↗

First page shown at left

Primary source: https://arxiv.org/abs/2201.11903

Chain-of-Thought: an external scratchpad

Intermediate tokens decompose one hard prediction into easier conditioned steps

Direct prompting

Q: Roger has 5 balls. He buys 2 cans of 3. How many now?

A: 11

No visible decomposition; weaker models often fail on multi-step tasks.

Chain-of-Thought

Roger starts with 5.
2 × 3 = 6 new balls.
5 + 6 = 11.

The generated steps become context for the next tokens.

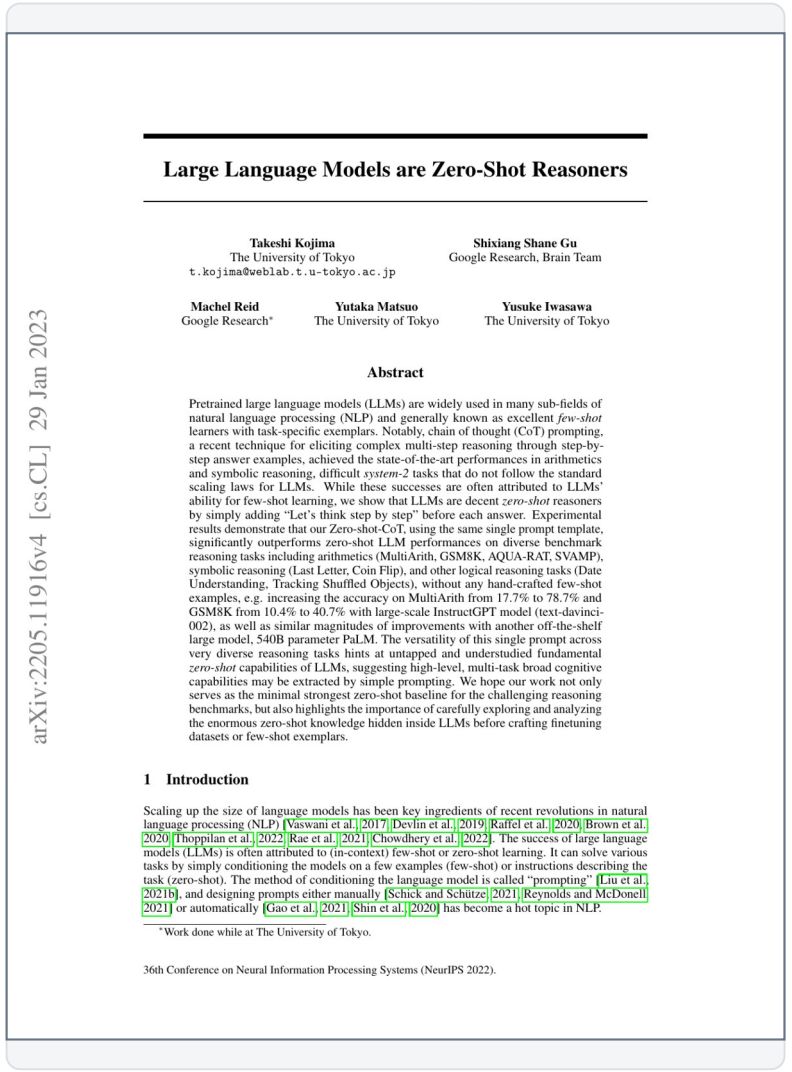
Elicits a capability; does not install one.

A plausible chain is not guaranteed faithful.

Source: Wei et al., Chain-of-Thought Prompting (NeurIPS 2022) • Week 2 notes §1.1

Paper spotlight: Zero-shot CoT

Large Language Models are Zero-Shot Reasoners • NeurIPS 2022



Large Language Models are Zero-Shot Reasoners

Takeshi Kojima
The University of Tokyo
t.kojima@weblab.t.u-tokyo.ac.jp

Shixiang Shane Gu
Google Research, Brain Team

Machel Reid
Google Research*

Yutaka Matsuo
The University of Tokyo

Yusuke Iwasawa
The University of Tokyo

Abstract

Pretrained large language models (LLMs) are widely used in many sub-fields of natural language processing (NLP) and generally known as excellent *few-shot* learners with task-specific exemplars. Notably, chain of thought (CoT) prompting, a recent technique for eliciting complex multi-step reasoning through step-by-step answer examples, achieved the state-of-the-art performances in arithmetics and symbolic reasoning, difficult *system-2* tasks that do not follow the standard scaling laws for LLMs. While these successes are often attributed to LLMs' ability for few-shot learning, we show that LLMs are decent *zero-shot* reasoners by simply adding "Let's think step by step" before each answer. Experimental results demonstrate that our Zero-shot-CoT, using the same single prompt template, significantly outperforms zero-shot LLM performances on diverse benchmark reasoning tasks including arithmetics (MultiArith, GSM8K, AQUA-RAT, SVAMP), symbolic reasoning (Last Letter, Coin Flip), and other logical reasoning tasks (Date Understanding, Tracking Shuffled Objects), without any hand-crafted few-shot examples, e.g. increasing the accuracy on MultiArith from 17.7% to 78.7% and GSM8K from 10.4% to 40.7% with large-scale InstructGPT model (text-davinci-002), as well as similar magnitudes of improvements with another off-the-shelf large model, 540B parameter PaLM. The versatility of this single prompt across very diverse reasoning tasks hints at untapped and understudied fundamental *zero-shot* capabilities of LLMs, suggesting high-level, multi-task broad cognitive capabilities may be extracted by simple prompting. We hope our work not only serves as the minimal strongest zero-shot baseline for the challenging reasoning benchmarks, but also highlights the importance of carefully exploring and analyzing the enormous zero-shot knowledge hidden inside LLMs before crafting finetuning datasets or few-shot exemplars.

1 Introduction

Scaling up the size of language models has been key ingredients of recent revolutions in natural language processing (NLP) [Vaswani et al., 2017, Devlin et al., 2019, Raffel et al., 2020, Brown et al., 2020, Thoppilan et al., 2022, Rae et al., 2021, Chowdhery et al., 2022]. The success of large language models (LLMs) is often attributed to (in-context) few-shot or zero-shot learning. It can solve various tasks by simply conditioning the models on a few examples (few-shot) or instructions describing the task (zero-shot). The method of conditioning the language model is called "prompting", [Liu et al., 2021b], and designing prompts either manually [Schick and Schütze, 2021, Reynolds and McDonell, 2021] or automatically [Gao et al., 2021, Shin et al., 2020] has become a hot topic in NLP.

*Work done while at The University of Tokyo.

36th Conference on Neural Information Processing Systems (NeurIPS 2022).

Main contribution

Demonstrates that a single generic instruction - 'Let's think step by step' - can elicit multi-step reasoning without hand-written exemplars across diverse tasks.

Intuition

The instruction selects a latent behavior the large model already possesses; it changes the generation mode without adding task-specific examples or updating weights.

Significance

Creates a strong, cheap baseline and separates the benefit of stepwise reasoning from the separate benefit of few-shot demonstrations.

Open paper ↗

First page shown at left

Primary source: https://arxiv.org/abs/2205.11916

Paper spotlight: Self-consistency

Self-Consistency Improves Chain of Thought Reasoning in Language Models • ICLR 2023

Published as a conference paper at ICLR 2023

SELF-CONSISTENCY IMPROVES CHAIN OF THOUGHT REASONING IN LANGUAGE MODELS

Xuezhi Wang[†‡] **Jason Wei**[†] **Dale Schuurmans**[†] **Quoc Le**[†] **Ed H. Chi**[†]
Sharan Narang[†] **Aakanksha Chowdhery**[†] **Denny Zhou**[†§]
[†]Google Research, Brain Team
[‡]xuezhiw@google.com, [§]dennyzhou@google.com

ABSTRACT

Chain-of-thought prompting combined with pre-trained large language models has achieved encouraging results on complex reasoning tasks. In this paper, we propose a new decoding strategy, *self-consistency*, to replace the naive greedy decoding used in chain-of-thought prompting. It first samples a diverse set of reasoning paths instead of only taking the greedy one, and then selects the most consistent answer by marginalizing out the sampled reasoning paths. Self-consistency leverages the intuition that a complex reasoning problem typically admits multiple different ways of thinking leading to its unique correct answer. Our extensive empirical evaluation shows that self-consistency boosts the performance of chain-of-thought prompting with a striking margin on a range of popular arithmetic and commonsense reasoning benchmarks, including GSM8K (+17.9%), SVAMP (+11.0%), AQuA (+12.2%), StrategyQA (+6.4%) and ARC-challenge (+3.9%).

1 INTRODUCTION

Although language models have demonstrated remarkable success across a range of NLP tasks, their ability to demonstrate reasoning is often seen as a limitation, which cannot be overcome solely by increasing model scale (Rae et al., 2021; BIG-bench collaboration, 2021, *inter alia*). In an effort to address this shortcoming, Wei et al. (2022) have proposed *chain-of-thought prompting*, where a language model is prompted to generate a series of short sentences that mimic the reasoning process a person might employ in solving a task. For example, given the question *"If there are 3 cars in the parking lot and 2 more cars arrive, how many cars are in the parking lot?"*, instead of directly responding with *"5"*, a language model would be prompted to respond with the entire chain-of-thought: *"There are 3 cars in the parking lot already. 2 more arrive. Now there are 3 + 2 = 5 cars. The answer is 5."*. It has been observed that chain-of-thought prompting significantly improves model performance across a variety of multi-step reasoning tasks (Wei et al., 2022).

In this paper, we introduce a novel decoding strategy called *self-consistency* to replace the greedy decoding strategy used in chain-of-thought prompting (Wei et al., 2022), that further improves language models' reasoning performance by a significant margin. Self-consistency leverages the intuition that complex reasoning tasks typically admit multiple reasoning paths that reach a correct answer (Stanovich & West, 2000). The more that deliberate thinking and analysis is required for a problem (Evans, 2010), the greater the diversity of reasoning paths that can recover the answer.

Figure 1 illustrates the self-consistency method with an example. We first prompt the language model with chain-of-thought prompting, then instead of greedily decoding the optimal reasoning path, we propose a "sample-and-marginalize" decoding procedure: we first *sample* from the language model's decoder to generate a *diverse* set of reasoning paths; each reasoning path might lead to a different final answer, so we determine the optimal answer by *marginalizing out* the sampled reasoning paths to find the most consistent answer in the final answer set. Such an approach is analogous to the human experience that if multiple different ways of thinking lead to the same answer, one has greater confidence that the final answer is correct. Compared to other decoding methods, self-consistency avoids the repetitiveness and local-optimality that plague greedy decoding, while mitigating the stochasticity of a single sampled generation.

1

Main contribution

Replaces greedy decoding with diverse sampled reasoning paths and selects the answer that is most consistent after marginalizing over those paths.

Intuition

A problem may support several valid reasoning routes. Wrong paths often disagree, while independently generated correct paths tend to converge on the same final answer.

Significance

Made test-time compute a visible reasoning lever: accuracy can improve through decoding and aggregation, but the benefit must be judged against roughly N-times cost and latency.

Open paper ↗

First page shown at left

Primary source: https://arxiv.org/abs/2203.11171

Two extensions: trigger or sample

Zero-shot CoT changes the prompt; self-consistency spends more test-time compute

Zero-shot CoT

Append “Let’s think step by step.”

No hand-written demonstrations. It separates the benefit of stepwise reasoning from the benefit of few-shot exemplars.

Limit: still depends on latent model capability.

Self-consistency

Sample N independent chains at temperature > 0, then majority-vote the final answers.

Benefit: uncorrelated errors can cancel.

Cost: roughly N× tokens and latency for sub-linear accuracy gain.

First recurring trade-off of the course: spend inference-time compute for accuracy.

Sources: Kojima et al. (2022) • Wang et al., Self-Consistency (ICLR 2023) • Week 2 notes §1.2

Paper spotlight: ReAct

ReAct: Synergizing Reasoning and Acting in Language Models • ICLR 2023

Published as a conference paper at ICLR 2023

REACT: SYNERGIZING REASONING AND ACTING IN LANGUAGE MODELS

Shunyu Yao*,1, Jeffrey Zhao[2], Dian Yu[2], Nan Du[2], Izhak Shafran[2], Karthik Narasimhan[1], Yuan Cao[2]

[1]Department of Computer Science, Princeton University
[2]Google Research, Brain team
[1]{shunyuy,karthikn}@princeton.edu
[2]{jeffreyzhao,dianyu,dunan,izhak,yuancao}@google.com

ABSTRACT

While large language models (LLMs) have demonstrated impressive performance across tasks in language understanding and interactive decision making, their abilities for reasoning (e.g. chain-of-thought prompting) and acting (e.g. action plan generation) have primarily been studied as separate topics. In this paper, we explore the use of LLMs to generate both reasoning traces and task-specific actions in an interleaved manner, allowing for greater synergy between the two: reasoning traces help the model induce, track, and update action plans as well as handle exceptions, while actions allow it to interface with and gather additional information from external sources such as knowledge bases or environments. We apply our approach, named ReAct, to a diverse set of language and decision making tasks and demonstrate its effectiveness over state-of-the-art baselines in addition to improved human interpretability and trustworthiness. Concretely, on question answering (HotpotQA) and fact verification (Fever), ReAct overcomes prevalent issues of hallucination and error propagation in chain-of-thought reasoning by interacting with a simple Wikipedia API, and generating human-like task-solving trajectories that are more interpretable than baselines without reasoning traces. Furthermore, on two interactive decision making benchmarks (ALFWorld and WebShop), ReAct outperforms imitation and reinforcement learning methods by an absolute success rate of 34% and 10% respectively, while being prompted with only one or two in-context examples.

1 INTRODUCTION

A unique feature of human intelligence is the ability to seamlessly combine task-oriented actions with verbal reasoning (or inner speech, Alderson-Day & Fernyhough, 2015), which has been theorized to play an important role in human cognition for enabling self-regulation or strategization (Vygotsky, 1987; Luria, 1965; Fernyhough, 2010) and maintaining a working memory (Baddeley, 1992). Consider the example of cooking up a dish in the kitchen. Between any two specific actions, we may reason in language in order to track progress ("now that everything is cut, I should heat up the pot of water"), to handle exceptions or adjust the plan according to the situation ("I don't have salt, so let me use soy sauce and pepper instead"), and to realize when external information is needed ("how do I prepare dough? Let me search on the Internet"). We may also act (open a cookbook to read the recipe, open the fridge, check ingredients) to support the reasoning and to answer questions ("What dish can I make right now?"). This tight synergy between "acting" and "reasoning" allows humans to learn new tasks quickly and perform robust decision making or reasoning, even under previously unseen circumstances or facing information uncertainties.

Recent results have hinted at the possibility of combining verbal reasoning with interactive decision making in autonomous systems. On one hand, properly prompted large language models (LLMs) have demonstrated emergent capabilities to carry out several steps of reasoning traces to derive

*Work during Google internship. Projet page with code: https://react-lm.github.io/

1

Main contribution

Interleaves free-text reasoning traces with task-specific actions and feeds environment observations back into the model before the next decision.

Intuition

Thoughts maintain and revise a plan; actions obtain external evidence. Each side corrects a weakness of the other: ungrounded reasoning and directionless acting.

Significance

Established the canonical Thought-Action-Observation agent loop used by many modern agent frameworks and made trajectories easier to inspect and critique.

Open paper ↗

First page shown at left

Primary source: https://arxiv.org/abs/2210.03629

ReAct: thought → action → observation

Reasoning becomes grounded because the environment can change the next decision

THOUGHT 1	Find the film that won Best Picture in 2023.
ACTION 1	`Search["2023 Best Picture winner"]`
OBSERVATION 1	Everything Everywhere All at Once won.
THOUGHT 2	Now identify the director(s).
ACTION 2	`Search["Everything Everywhere… director"]`
OBSERVATION 2	Daniel Kwan and Daniel Scheinert.

The thought is not executed-it exists to condition the next action.

Source: Yao et al., ReAct (ICLR 2023) • Week 2 notes §2.1 • Course website worked trace

ReAct's hidden assumptions

A mechanism becomes critique when we name what must be true

CHEAP + FAST

Search-like actions cost little and return quickly.

IF FALSE

Paid, destructive, or multi-minute actions change the trade-off.

SMALL ACTION SPACE

A few named tools have clear semantics.

IF FALSE

Dozens or hundreds of tools create a selection problem.

TRUTHFUL OBSERVATIONS

Tool results are immediate, complete, and benign.

IF FALSE

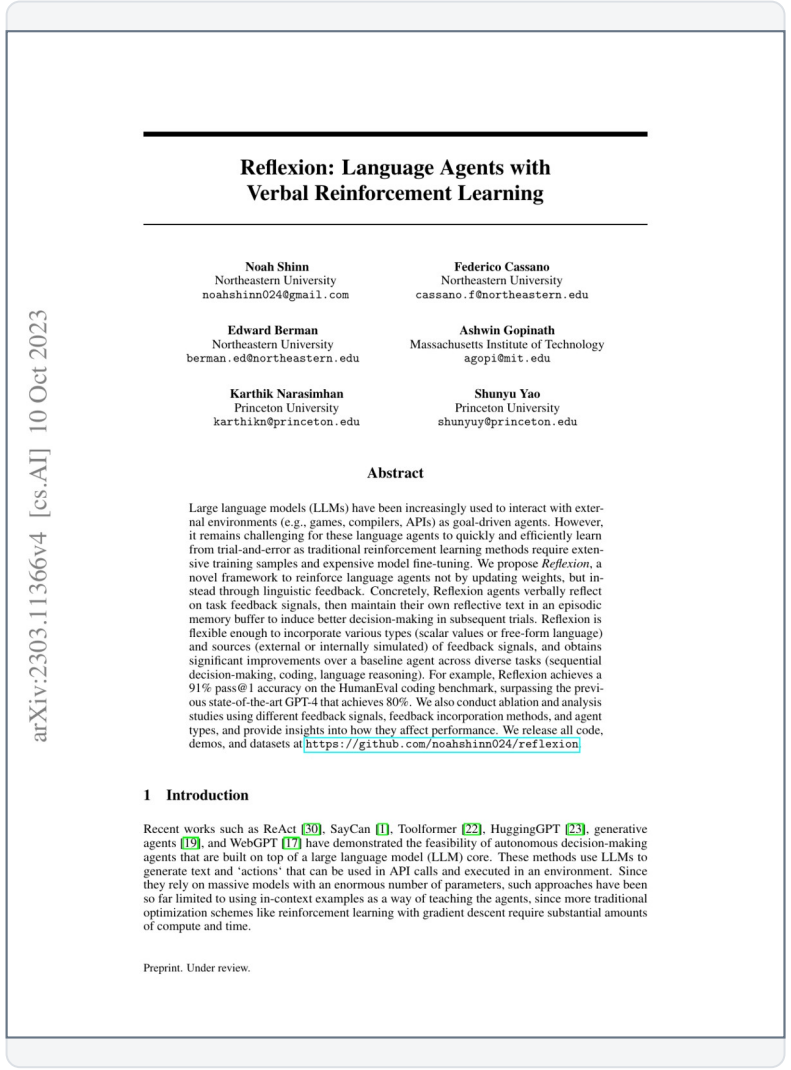
Attacker-controlled results turn the feedback edge into prompt injection.

Later sessions: irreversibility • tool selection • indirect prompt injection

Source: Week 2 notes §2.2

Paper spotlight: Reflexion

Reflexion: Language Agents with Verbal Reinforcement Learning • NeurIPS 2023



Reflexion: Language Agents with Verbal Reinforcement Learning

Noah Shinn
Northeastern University
noahshinn024@gmail.com

Federico Cassano
Northeastern University
cassano.f@northeastern.edu

Edward Berman
Northeastern University
berman.ed@northeastern.edu

Ashwin Gopinath
Massachusetts Institute of Technology
agopi@mit.edu

Karthik Narasimhan
Princeton University
karthikn@princeton.edu

Shunyu Yao
Princeton University
shunyuy@princeton.edu

Abstract

Large language models (LLMs) have been increasingly used to interact with external environments (e.g., games, compilers, APIs) as goal-driven agents. However, it remains challenging for these language agents to quickly and efficiently learn from trial-and-error as traditional reinforcement learning methods require extensive training samples and expensive model fine-tuning. We propose *Reflexion*, a novel framework to reinforce language agents not by updating weights, but instead through linguistic feedback. Concretely, Reflexion agents verbally reflect on task feedback signals, then maintain their own reflective text in an episodic memory buffer to induce better decision-making in subsequent trials. Reflexion is flexible enough to incorporate various types (scalar values or free-form language) and sources (external or internally simulated) of feedback signals, and obtains significant improvements over a baseline agent across diverse tasks (sequential decision-making, coding, language reasoning). For example, Reflexion achieves a 91% pass@1 accuracy on the HumanEval coding benchmark, surpassing the previous state-of-the-art GPT-4 that achieves 80%. We also conduct ablation and analysis studies using different feedback signals, feedback incorporation methods, and agent types, and provide insights into how they affect performance. We release all code, demos, and datasets at https://github.com/noahshinn024/reflexion.

1 Introduction

Recent works such as ReAct [30], SayCan [1], Toolformer [22], HuggingGPT [23], generative agents [19], and WebGPT [17] have demonstrated the feasibility of autonomous decision-making agents that are built on top of a large language model (LLM) core. These methods use LLMs to generate text and 'actions' that can be used in API calls and executed in an environment. Since they rely on massive models with an enormous number of parameters, such approaches have been so far limited to using in-context examples as a way of teaching the agents, since more traditional optimization schemes like reinforcement learning with gradient descent require substantial amounts of compute and time.

Preprint. Under review.

Main contribution

Uses feedback to generate a natural-language reflection, stores it in episodic memory, and conditions later attempts on that reflection without updating model weights.

Intuition

Turn a failed trajectory into a concise lesson that remains in context for the next trial, so the agent can avoid repeating the same mistake.

Significance

Popularized verbal self-improvement for agents and exposed a central dependency that recurs throughout the course: the reliability of the evaluator producing the feedback.

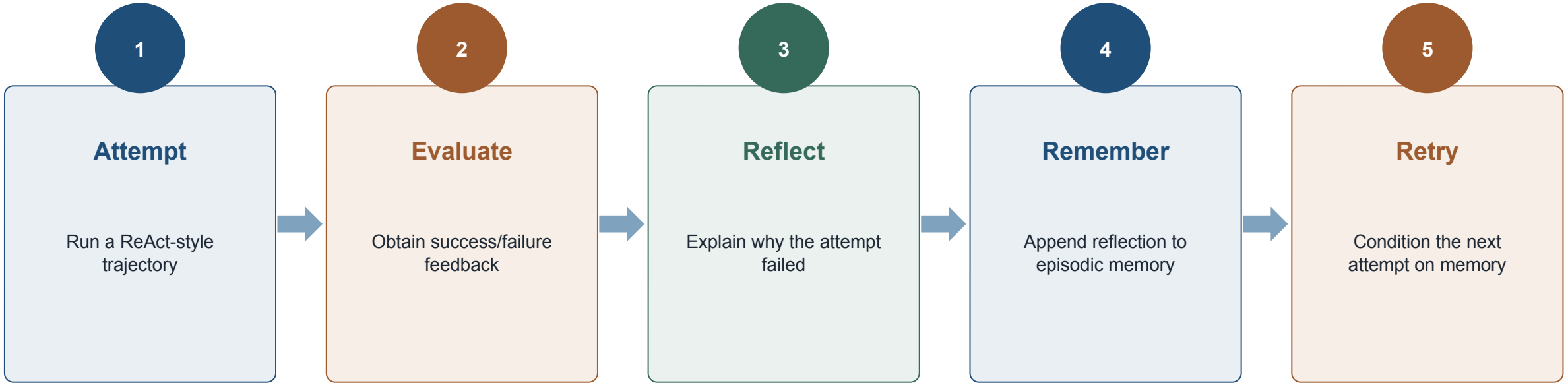
Open paper ↗

First page shown at left

Primary source: https://arxiv.org/abs/2303.11366

Reflexion: retry with verbal memory

An evaluator turns a failed trajectory into a reflection for the next attempt



“Verbal RL” changes context, not model weights.

Source: Shinn et al., Reflexion (NeurIPS 2023) • Week 2 notes §3.1

Paper spotlight: Intrinsic self-correction

Large Language Models Cannot Self-Correct Reasoning Yet • ICLR 2024

Google DeepMind

LARGE LANGUAGE MODELS CANNOT SELF-CORRECT REASONING YET

Jie Huang[1,2*] **Xinyun Chen**[1*] **Swaroop Mishra**[1] **Huaixiu Steven Zheng**[1] **Adams Wei Yu**[1]
Xinying Song[1] **Denny Zhou**[1]
[1]Google DeepMind [2]University of Illinois at Urbana-Champaign
jeffhj@illinois.edu, {xinyunchen, dennyzhou}@google.com

ABSTRACT

Large Language Models (LLMs) have emerged as a groundbreaking technology with their unparalleled text generation capabilities across various applications. Nevertheless, concerns persist regarding the accuracy and appropriateness of their generated content. A contemporary methodology, *self-correction*, has been proposed as a remedy to these issues. Building upon this premise, this paper critically examines the role and efficacy of self-correction within LLMs, shedding light on its true potential and limitations. Central to our investigation is the notion of *intrinsic self-correction*, whereby an LLM attempts to correct its initial responses based solely on its inherent capabilities, without the crutch of external feedback. In the context of reasoning, our research indicates that LLMs struggle to self-correct their responses without external feedback, and at times, their performance even degrades after self-correction. Drawing from these insights, we offer suggestions for future research and practical applications in this field.

1 INTRODUCTION

The rapid advancements in the domain of artificial intelligence have ushered in the era of Large Language Models (LLMs). These models, characterized by their expansive parameter counts and unparalleled capabilities in text generation, have showcased promising results across a multitude of applications (Chowdhery et al., 2023; Anil et al., 2023; OpenAI, 2023; *inter alia*). However, concerns about their accuracy, reasoning capabilities, and the safety of their generated content have drawn significant attention from the community (Bang et al., 2023; Alkaissi & McFarlane, 2023; Zheng et al., 2023; Shi et al., 2023; Carlini et al., 2021; Huang et al., 2022; Shao et al., 2023; Li et al., 2023; Wei et al., 2023; Zhou et al., 2023b; Zou et al., 2023; *inter alia*).

Amidst this backdrop, the concept of "self-correction" has emerged as a promising solution, where LLMs refine their responses based on feedback to their previous outputs (Madaan et al., 2023; Welleck et al., 2023; Shinn et al., 2023; Kim et al., 2023; Bai et al., 2022; Ganguli et al., 2023; Gao et al., 2023; Paul et al., 2023; Chen et al., 2023b; Pan et al., 2023; *inter alia*). However, the underlying mechanics and efficacy of self-correction in LLMs remain underexplored. A fundamental question arises: If an LLM possesses the ability to self-correct, why doesn't it simply offer the correct answer in its initial attempt? This paper delves deeply into this paradox, critically examining the self-correction capabilities of LLMs, with a particular emphasis on reasoning (Wei et al., 2022; Zhou et al., 2023b; Huang & Chang, 2023).

To study this, we first define the concept of *intrinsic self-correction*, a scenario wherein the model endeavors to rectify its initial responses based solely on its inherent capabilities, without the crutch of external feedback. Such a setting is crucial because high-quality external feedback is often unavailable in many real-world applications. Moreover, it is vital to understand the intrinsic capabilities of LLMs. Contrary to the optimism surrounding self-correction (Madaan et al., 2023; Kim et al., 2023; Shinn et al., 2023; Pan et al., 2023; *inter alia*), our findings indicate that LLMs struggle to self-correct their reasoning in this setting. In most instances, the performance after self-correction

*Equal contribution.

1

Main contribution

Systematically tests intrinsic self-correction - revision without external feedback - and finds that models often fail to improve and can even reduce reasoning accuracy.

Intuition

The same model cannot reliably decide whether its unsupported answer is wrong; a fluent self-critique is not an independent source of evidence.

Significance

Raises the evidence bar for reflection claims and forces a distinction between feedback-grounded retry and a model merely reconsidering its own output.

Open paper ↗

First page shown at left

Primary source: https://arxiv.org/abs/2310.01798

Feedback before reflection

A self-improvement claim is only as strong as its evaluator

1 External ground truth

Unit tests; environment success signal

Strongest

2 External heuristic

LLM judge or rule-based score

Useful, biased

3 Same model, no signal

Model judges its own unsupported answer

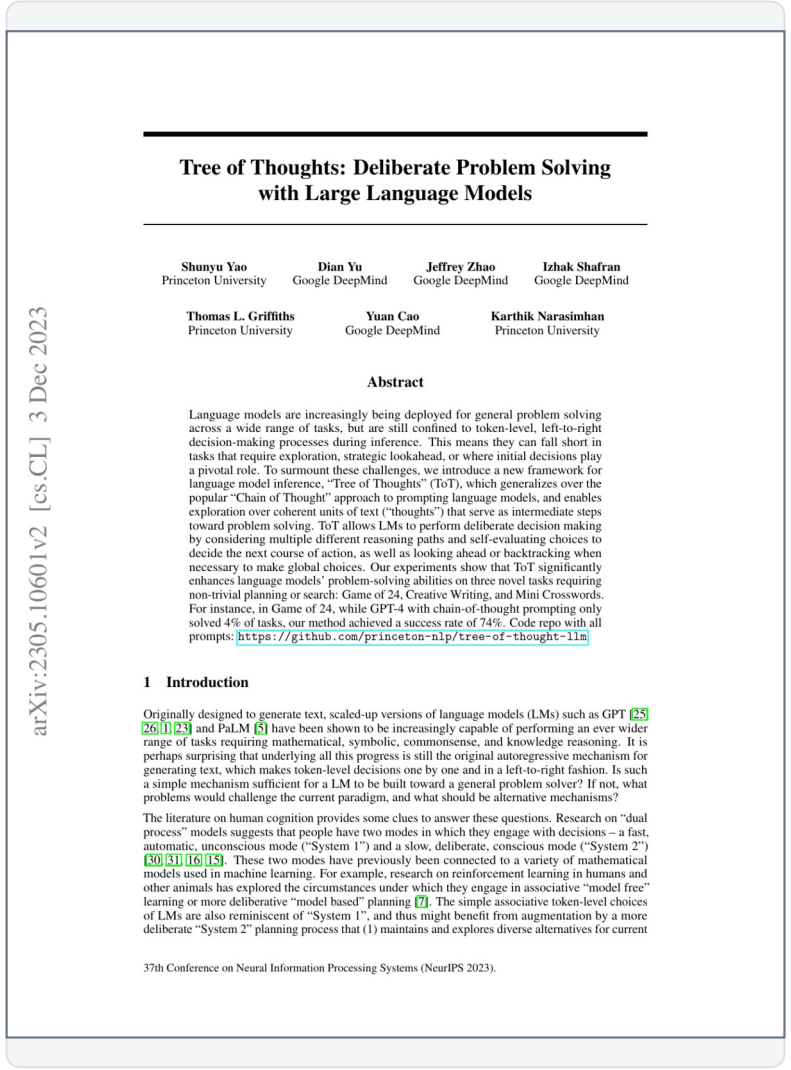
Unreliable

Ask immediately: which case is the paper-or your project-actually using?

Sources: Week 2 notes §3.2 • Huang et al., “LLMs Cannot Self-Correct Reasoning Yet” (ICLR 2024)

Paper spotlight: Tree-of-Thoughts

Tree of Thoughts: Deliberate Problem Solving with Large Language Models • NeurIPS 2023



Tree of Thoughts: Deliberate Problem Solving with Large Language Models

Shunyu Yao Princeton University
Dian Yu Google DeepMind
Jeffrey Zhao Google DeepMind
Izhak Shafran Google DeepMind
Thomas L. Griffiths Princeton University
Yuan Cao Google DeepMind
Karthik Narasimhan Princeton University

Abstract

Language models are increasingly being deployed for general problem solving across a wide range of tasks, but are still confined to token-level, left-to-right decision-making processes during inference. This means they can fall short in tasks that require exploration, strategic lookahead, or where initial decisions play a pivotal role. To surmount these challenges, we introduce a new framework for language model inference, "Tree of Thoughts" (ToT), which generalizes over the popular "Chain of Thought" approach to prompting language models, and enables exploration over coherent units of text ("thoughts") that serve as intermediate steps toward problem solving. ToT allows LMs to perform deliberate decision making by considering multiple different reasoning paths and self-evaluating choices to decide the next course of action, as well as looking ahead or backtracking when necessary to make global choices. Our experiments show that ToT significantly enhances language models' problem-solving abilities on three novel tasks requiring non-trivial planning or search: Game of 24, Creative Writing, and Mini Crosswords. For instance, in Game of 24, while GPT-4 with chain-of-thought prompting only solved 4% of tasks, our method achieved a success rate of 74%. Code repo with all prompts: https://github.com/princeton-nlp/tree-of-thought-llm

1 Introduction

Originally designed to generate text, scaled-up versions of language models (LMs) such as GPT [25, 26, 1, 23] and PaLM [5] have been shown to be increasingly capable of performing an ever wider range of tasks requiring mathematical, symbolic, commonsense, and knowledge reasoning. It is perhaps surprising that underlying all this progress is still the original autoregressive mechanism for generating text, which makes token-level decisions one by one and in a left-to-right fashion. Is such a simple mechanism sufficient for a LM to be built toward a general problem solver? If not, what problems would challenge the current paradigm, and what should be alternative mechanisms?

The literature on human cognition provides some clues to answer these questions. Research on "dual process" models suggests that people have two modes in which they engage with decisions – a fast, automatic, unconscious mode ("System 1") and a slow, deliberate, conscious mode ("System 2") [30, 31, 16, 15]. These two modes have previously been connected to a variety of mathematical models used in machine learning. For example, research on reinforcement learning in humans and other animals has explored the circumstances under which they engage in associative "model free" learning or more deliberative "model based" planning [7]. The simple associative token-level choices of LMs are also reminiscent of "System 1", and thus might benefit from augmentation by a more deliberate "System 2" planning process that (1) maintains and explores diverse alternatives for current

37th Conference on Neural Information Processing Systems (NeurIPS 2023).

Main contribution

Generalizes a single reasoning chain into search over coherent thought units using a generator, state evaluator, search algorithm, and stopping rule.

Intuition

Delay commitment: explore several partial solutions, score their promise, backtrack from weak choices, and continue from stronger states.

Significance

Reframed inference as explicit search and made two hidden costs visible - the number of model calls and the reliability of the evaluator guiding pruning.

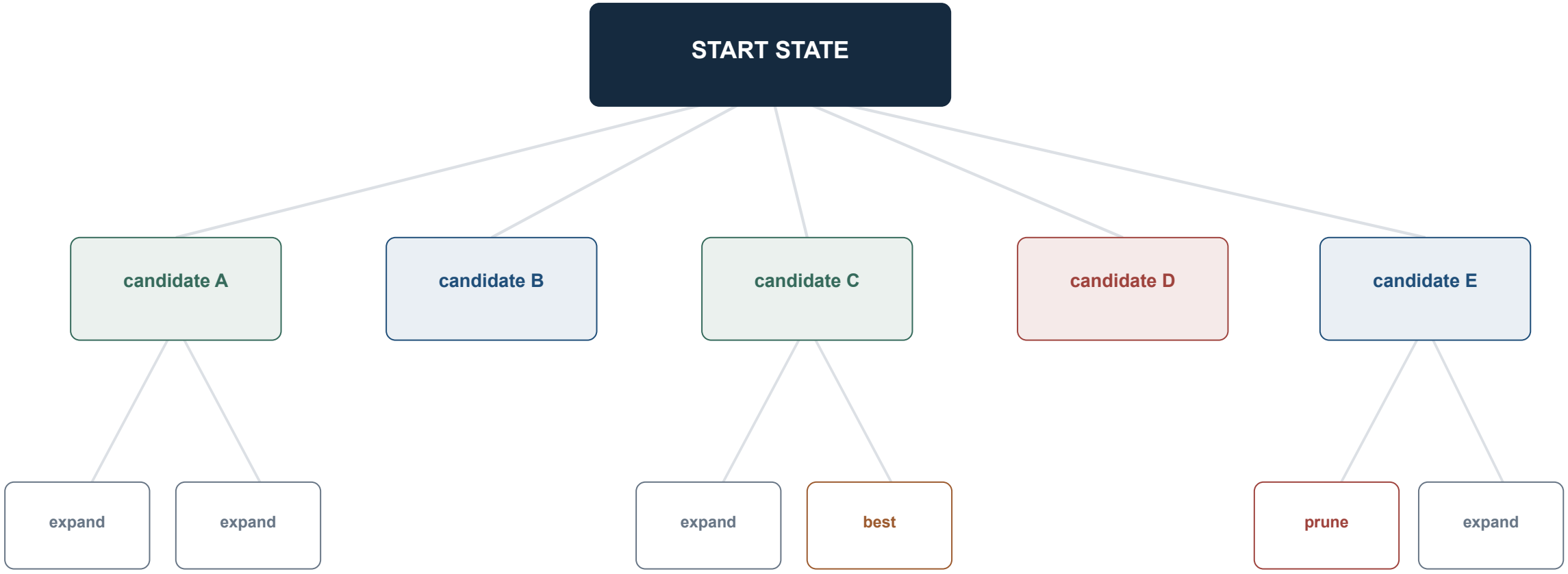
Open paper ↗

First page shown at left

Primary source: https://arxiv.org/abs/2305.10601

Tree-of-Thoughts: reason by search

Generate, evaluate, expand, prune-and stop



Generator

propose candidates

Evaluator

score partial states

Search

BFS / DFS / MCTS

Stopping

solution or budget

Source: Yao et al., Tree of Thoughts (NeurIPS 2023) • Week 2 notes §4.1

Paper spotlight: Graph-of-Thoughts

Graph of Thoughts: Solving Elaborate Problems with Large Language Models • AAAI 2024

Graph of Thoughts: Solving Elaborate Problems with Large Language Models

Maciej Besta[1*], Nils Blach[1*], Ales Kubicek[1], Robert Gerstenberger[1], Michał Podstawski[2], Lukas Gianinazzi[1], Joanna Gajda[3], Tomasz Lehmann[3], Hubert Niewiadomski[3], Piotr Nyczyk[3], Torsten Hoefler[1]
[1]ETH Zurich, [2]Warsaw University of Technology, [3]Cledar
bestam@inf.ethz.ch, nils.blach@inf.ethz.ch, htor@inf.ethz.ch

Abstract

We introduce Graph of Thoughts (GoT): a framework that advances prompting capabilities in large language models (LLMs) beyond those offered by paradigms such as Chain-of-Thought or Tree of Thoughts (ToT). The key idea and primary advantage of GoT is the ability to model the information generated by an LLM as an *arbitrary graph*, where units of information ("LLM thoughts") are vertices, and edges correspond to dependencies between these vertices. This approach enables combining arbitrary LLM thoughts into synergistic outcomes, distilling the essence of whole networks of thoughts, or enhancing thoughts using feedback loops. We illustrate that GoT offers advantages over state of the art on different tasks, for example increasing the quality of sorting by 62% over ToT, while simultaneously reducing costs by >31%. We ensure that GoT is extensible with new thought transformations and thus can be used to spearhead new prompting schemes. This work brings the LLM reasoning closer to human thinking or brain mechanisms such as recurrence, both of which form complex networks.

Website & code: https://github.com/spcl/graph-of-thoughts

1 Introduction

Large language models (LLMs) are taking over the world of AI. Recent years saw a rapid development of models primarily based on the decoder-only Transformer variant [65], such as GPT [13, 14, 53, 54], PaLM [19], or LLaMA [63].

Prompt engineering is a resource-efficient approach for solving different LLM tasks. In brief, one includes the task description within the input sent to an LLM. If this description is appropriately formulated, the LLM solves the task using its autoregressive token-based mechanism for generating text. Such prompts may contain example tasks with solutions (few-shot prompting, also referred to as in-context learning (ICL)), or even no example tasks at all (zero-shot prompting). In recent years it was shown that this mechanism can be used to solve a broad set of tasks that involve mathematical, commonsense, or symbolic reasoning.

Chain-of-Thought (CoT) [71] is an approach for prompting, in which one includes the intermediate steps of reasoning within the prompt (intermediate "thoughts"), besides the task input/output. CoT was shown to significantly improve the capability of LLMs to solve problems without resorting to any model updates. One major improvement over CoT, Self-Consistency with CoT (CoT-SC) [67], is a scheme where multiple CoTs are generated, and then the best one is selected as the outcome. More recently, CoT and CoT-SC were extended with Tree of Thoughts (ToT) [43, 75, 77], which models the LLM reasoning process with a tree. This facilitates using different paths of thoughts, and offers novel capabilities such as backtracking from non-promising outcomes. Unfortunately, the ToT approaches still fundamentally limit the reasoning abilities within a prompt by imposing the rigid tree structure on the thought process.

In this work, we argue that fundamentally more powerful prompting can be achieved by enabling LLM thoughts to form an arbitrary graph structure. This is motivated by numerous phenomena such as human reasoning, brain structure, or algorithmic execution. When working on a novel idea, a human would not only follow a chain of thoughts (as in CoT) or try different separate ones (as in ToT), but would actually form a more complex network of thoughts. For example, one could explore a certain chain of reasoning, backtrack and start a new one, then realize that a certain idea from the previous chain could be combined with the currently explored one, and merge them both into a new solution, taking advantage of their strengths and eliminating their weaknesses. Similarly, brains form complex networks, with graph-like patterns such as recurrence [28]. Executing algorithms also expose networked patterns, often represented by Directed Acyclic Graphs. The corresponding *graph-enabled transformations* bring a promise of more powerful prompting when applied to LLM thoughts, but they are not naturally expressible with CoT or ToT.

We observe that these (and many other) thought transformations can be naturally enabled when *modeling the reasoning process of an LLM as a graph*. For this, we propose Graph of Thoughts (GoT), an approach that *enhances LLMs' capabilities through networked reasoning* (**contribution #1**). In GoT, an LLM thought is modeled as a vertex, while an edge is a dependency between such thoughts. Using GoT, one can aggregate arbitrary thoughts by constructing vertices that have more than one incoming edge. Overall, the graph abstraction harnessed by GoT seamlessly generalizes CoT and ToT to more complex thought patterns, *without resorting to any model updates*.

Yet, putting GoT to practice requires solving several design challenges. For example, what is the best graph structure for different tasks? How to best aggregate thoughts to maximize accuracy and minimize cost? To answer these and

*Equal contribution

Main contribution

Represents generated thoughts as an arbitrary graph, enabling aggregation, merging, refinement, and feedback operations beyond a tree's isolated branches.

Intuition

Good solutions may be assembled from several imperfect partial ideas; a graph allows those ideas to interact instead of forcing one branch to win and the rest to disappear.

Significance

Expanded the design space for reasoning orchestration while creating a sharper evaluation question: when does extra structure improve quality enough to justify added prompts and control logic?

Open paper ↗

First page shown at left

Primary source: https://arxiv.org/abs/2308.09687

Paper spotlight: LATS

Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models • ICML 2024

Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models

Andy Zhou [1 2] Kai Yan [1] Michal Shlapentokh-Rothman [1] Haohan Wang [1] Yu-Xiong Wang [1]

Abstract

While language models (LMs) have shown potential across a range of decision-making tasks, their reliance on simple acting processes limits their broad deployment as autonomous agents. In this paper, we introduce Language Agent Tree Search (LATS) – *the first general* framework that *synergizes* the capabilities of LMs in reasoning, acting, and planning. By leveraging the in-context learning ability of LMs, we integrate Monte Carlo Tree Search into LATS to enable LMs as agents, along with LM-powered value functions and self-reflections for proficient exploration and enhanced decision-making. A key feature of our approach is the incorporation of an environment for external feedback, which offers a more deliberate and adaptive problem-solving mechanism that surpasses the constraints of existing techniques. Our experimental evaluation across diverse domains, including programming, interactive question-answering (QA), web navigation, and math, validates the effectiveness and generality of LATS in decision-making while maintaining competitive or improved reasoning performance. Notably, LATS achieves state-of-the-art pass@1 accuracy (92.7%) for programming on HumanEval with GPT-4 and demonstrates gradient-free performance (average score of 75.9) comparable to gradient-based fine-tuning for web navigation on WebShop with GPT-3.5. Code can be found at https://github.com/lapisrocks/LanguageAgentTreeSearch.

Figure 1. Overview of LATS. Serving as a unified framework, LATS leverages an external environment and an MCTS-based search algorithm to improve reasoning and decision-making.

1. Introduction

General autonomous agents capable of reasoning and decision-making in a variety of environments (Wooldridge and Jennings, 1995) have been of longstanding interest in the field of artificial intelligence. While this has traditionally been studied in reinforcement learning, the recent rise of language models (LMs) (Brown et al., 2020; Chowdhery et al., 2023; Touvron et al., 2023; OpenAI, 2023) with strong reasoning and general adaptability offers an alternative paradigm. Not only have LMs excelled in standard natural language processing (NLP) tasks such as summarization (Nallapati et al., 2016) and language inference (Bowman et al., 2015), but they have also been adapted to an increasingly diverse set of tasks that often require advanced common-sense reasoning or quantitative skills (Cobbe et al., 2021; Saparov and He, 2023). In addition, LMs are capable of performing in complex environments that involve knowledge and reasoning, such as web navigation (Yao et al., 2022; Deng et al., 2023), tool-use (Schick et al., 2023), and open-ended games (Fan et al., 2022).

Reasoning and acting abilities have been further improved by prompting techniques that augment LMs with feedback or observations from an external environment, as exemplified by ReAct (Yao et al., 2023b) and other work (Gao et al., 2023; Shinn et al., 2023). This eliminates the need to rely entirely on the base abilities of LMs, enhancing them through external tools or semantic feedback. Despite such strengths, these methods are reflexive and fall short of humans' deliberate and thoughtful decision-making characteristics to solve problems (Sloman, 1996; Evans, 2010). In particular, they fail to consider multiple reasoning paths or to plan ahead. Recent search-guided LM work (Xie et al., 2023; Yao et al., 2023a; Hao et al., 2023) addresses this issue by searching over multiple reasoning chains. While enabling planning,

[1]University of Illinois Urbana-Champaign. [2]Lapis Labs. Correspondence to: Andy Zhou <andyz3@illinois.edu>.

Proceedings of the 41st International Conference on Machine Learning, Vienna, Austria. PMLR 235, 2024.

Main contribution

Integrates Monte Carlo Tree Search with LM value estimates, self-reflection, actions, and environment feedback to search over full agent trajectories.

Intuition

Use ReAct-style interaction to ground candidate paths, then spend search effort on trajectories whose observations and estimated value make them promising.

Significance

Unifies reasoning, acting, and planning in one search framework and shows the potential of grounded feedback, while occupying the highest-cost end of the comparison.

Open paper ↗

First page shown at left

Primary source: https://arxiv.org/abs/2310.04406

Search helps only if evaluation helps

ToT, GoT, and LATS change the structure and grounding of exploration

Tree-of-Thoughts

Branches over partial solutions.

Evaluator: usually another LLM call.

Risk: a poor evaluator prunes good paths early.

Cost: O(branches × depth) calls.

Graph-of-Thoughts

Adds merging and refinement across partial solutions.

Useful when the answer combines imperfect pieces.

Risk: more orchestration for potentially marginal gains.

LATS

Runs MCTS over ReAct-style trajectories.

Evaluator: real observations + value estimates.

Benefit: more grounded feedback.

Cost: highest in this family.

Critical question: is the gain from search-or from the evaluator that guides it?

Sources: Yao et al. (2023) • Besta et al. (2024) • Zhou et al., LATS (ICML 2024) • Week 2 notes §4

COMPARISON

Reasoning methods: cost and feedback

Choose the simplest row that fits the task and its available signal

METHOD	STRUCTURE	FEEDBACK	RELATIVE COST	BEST WHEN
CoT	linear chain	none	~1× longer	stepwise decomposition helps
Self-consistency	N chains + vote	none	N×	errors are diverse
ReAct	chain + actions	tool observations	~k steps	external info/actions needed
Reflexion	trials + memory	evaluator	trials ×	cheap ground truth exists
ToT / GoT	branching search	LLM evaluator	branches × depth	partials can be judged
LATS	MCTS trajectories	observations + value	highest	grounded high-stakes search

Source: Week 2 notes §4.3

Planning ≠ a plausible checklist

Classical planning makes a stronger, verifiable claim than natural-language decomposition

Task decomposition

Produces an ordered or partially ordered list of subtasks in natural language.

Useful: reduces cognitive load and organizes execution.

Missing: formal states, preconditions, effects, and correctness guarantees.

Classical planning

Given a world model, actions, preconditions, effects, initial state, and goal-search for a valid action sequence.

May offer soundness, completeness, or optimality guarantees.

Requires formal verification against the model.

Whenever a paper says “plans,” ask which definition it means.

Source: Week 2 notes §5.1

Paper spotlight: Plan-and-Solve

Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning • ACL 2023

Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models

Lei Wang[1] Wanyu Xu[2] Yihuai Lan Zhiqiang Hu[3] Yunshi Lan[4]
Roy Ka-Wei Lee[3] Ee-Peng Lim[1]*
[1]Singapore Management University
[2]Southwest Jiaotong University
[3]Singapore University of Technology and Design
[4]East China Normal University

Abstract

Large language models (LLMs) have recently been shown to deliver impressive performance in various NLP tasks. To tackle multi-step reasoning tasks, few-shot chain-of-thought (CoT) prompting includes a few manually crafted step-by-step reasoning demonstrations which enable LLMs to explicitly generate reasoning steps and improve their reasoning task accuracy. To eliminate the manual effort, Zero-shot-CoT concatenates the target problem statement with "*Let's think step by step*" as an input prompt to LLMs. Despite the success of Zero-shot-CoT, it still suffers from three pitfalls: calculation errors, missing-step errors, and semantic misunderstanding errors. To address the missing-step errors, we propose Plan-and-Solve (PS) Prompting. It consists of two components: first, devising a plan to divide the entire task into smaller subtasks, and then carrying out the subtasks according to the plan. To address the calculation errors and improve the quality of generated reasoning steps, we extend PS prompting with more detailed instructions and derive PS+ prompting. We evaluate our proposed prompting strategy on ten datasets across three reasoning problems. The experimental results over GPT-3 show that our proposed zero-shot prompting consistently outperforms Zero-shot-CoT across all datasets by a large margin, is comparable to or exceeds Zero-shot-Program-of-Thought Prompting, and has comparable performance with 8-shot CoT prompting on the math reasoning problem. The code can be found at https://github.com/AGI-Edgerunners/Plan-and-Solve-Prompting.

Figure 1: Error analysis of 46 GSM8K problems with incorrect answers returned by Zero-shot-CoT using GPT-3 LLM. Following Wei et al. (2022b) and Wang et al. (2022a), we assign "Calculation Error" (7%), "Step Missing Error" (12%), or "Semantic misunderstanding Error" (27%) to each incorrect answer.

1 Introduction

Large language models (LLMs) (Brown et al., 2020; Thoppilan et al., 2022; Chowdhery et al., 2022) have recently proven highly effective in various NLP tasks. Unlike the previous pre-trained language models (PTMs) (Devlin et al., 2019; Liu et al., 2019), these LLMs are typically provided as a service, with no access to model parameters due to commercial considerations and potential risks of misuse (Sun et al., 2022). Thus, it is challenging to fine-tune LLMs for downstream tasks (He et al., 2021; Houlsby et al., 2019; Devlin et al., 2019). Instead, we leverage LLMs to solve complex reasoning problems by eliciting their strong reasoning abilities over their embedded knowledge using instructions (or trigger sentences). So far, LLMs have shown impressive abilities to solve new reasoning problems by simply conditioning them on a few illustrative examples (i.e., few-shot learning) or a prompt to solve new problems without illustrative examples (i.e., zero-shot learning).

To tackle multi-step complex reasoning tasks using LLMs, Wei et al. (2022b) proposes few-shot chain-of-thought (CoT) prompting, which enables LLMs to explicitly generate the intermediate reasoning steps before predicting the final answer with a few manual step-by-step reasoning demonstration examples. In (Kojima et al., 2022), Zero-shot CoT eliminates the need for manually crafted examples in prompts by appending "*Let's think step by step*" to the target problem fed to LLMs such

*Corresponding author.

Main contribution

Separates an explicit planning phase from execution and adds detailed PS+ instructions to reduce missing-step and calculation errors in zero-shot reasoning.

Intuition

Decide the decomposition before solving: a complete subtask list makes omitted steps easier to avoid and creates a plan that can be inspected before execution begins.

Significance

Provides a simple ancestor of plan-and-execute agents, while exposing the trade-off between upfront reviewability and ReAct's ability to adapt the plan after each observation.

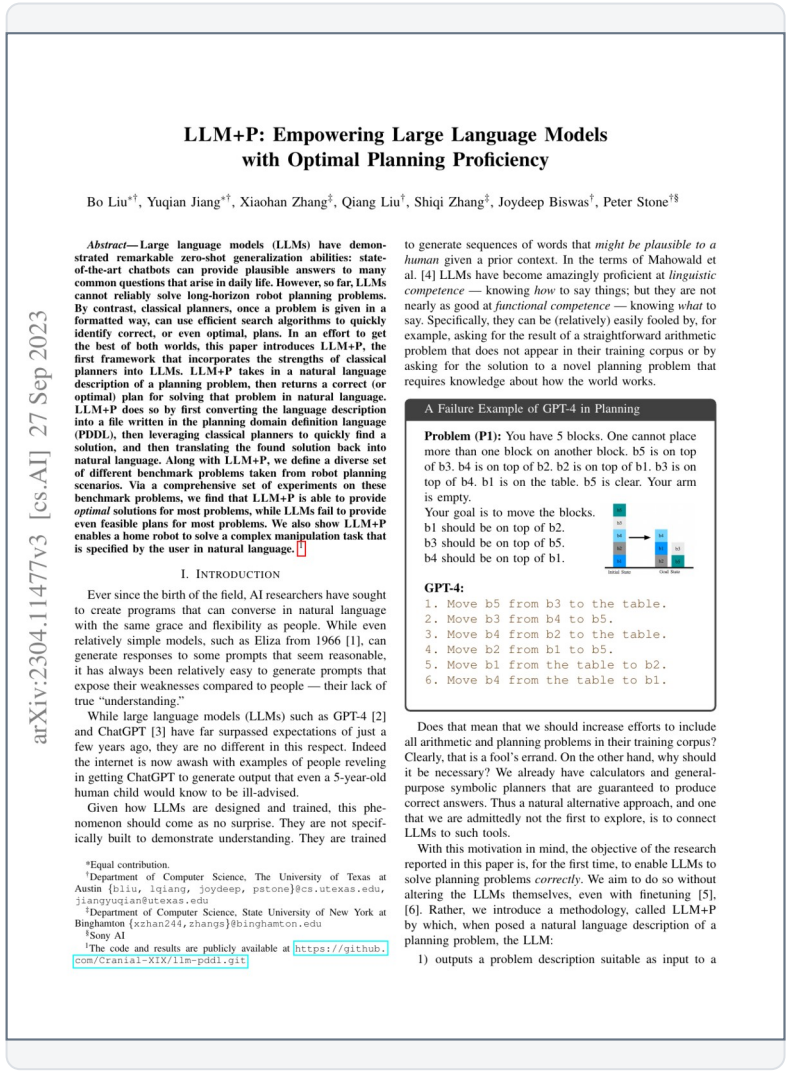
Open paper ↗

First page shown at left

Primary source: https://arxiv.org/abs/2305.04091

Paper spotlight: LLM+P

LLM+P: Empowering Large Language Models with Optimal Planning Proficiency • arXiv 2023



LLM+P: Empowering Large Language Models with Optimal Planning Proficiency

Bo Liu*†, Yuqian Jiang*†, Xiaohan Zhang‡, Qiang Liu†, Shiqi Zhang‡, Joydeep Biswas†, Peter Stone‡§

***Abstract*— Large language models (LLMs) have demonstrated remarkable zero-shot generalization abilities: state-of-the-art chatbots can provide plausible answers to many common questions that arise in daily life. However, so far, LLMs cannot reliably solve long-horizon robot planning problems. By contrast, classical planners, once a problem is given in a formatted way, can use efficient search algorithms to quickly identify correct, or even optimal, plans. In an effort to get the best of both worlds, this paper introduces LLM+P, the first framework that incorporates the strengths of classical planners into LLMs. LLM+P takes in a natural language description of a planning problem, then returns a correct (or optimal) plan for solving that problem in natural language. LLM+P does so by first converting the language description into a file written in the planning domain definition language (PDDL), then leveraging classical planners to quickly find a solution, and then translating the found solution back into natural language. Along with LLM+P, we define a diverse set of different benchmark problems taken from robot planning scenarios. Via a comprehensive set of experiments on these benchmark problems, we find that LLM+P is able to provide *optimal* solutions for most problems, while LLMs fail to provide even feasible plans for most problems. We also show LLM+P enables a home robot to solve a complex manipulation task that is specified by the user in natural language.** [1]

I. Introduction

Ever since the birth of the field, AI researchers have sought to create programs that can converse in natural language with the same grace and flexibility as people. While even relatively simple models, such as Eliza from 1966 [1], can generate responses to some prompts that seem reasonable, it has always been relatively easy to generate prompts that expose their weaknesses compared to people — their lack of true "understanding."

While large language models (LLMs) such as GPT-4 [2] and ChatGPT [3] have far surpassed expectations of just a few years ago, they are no different in this respect. Indeed the internet is now awash with examples of people reveling in getting ChatGPT to generate output that even a 5-year-old human child would know to be ill-advised.

Given how LLMs are designed and trained, this phenomenon should come as no surprise. They are not specifically built to demonstrate understanding. They are trained to generate sequences of words that *might be plausible to a human* given a prior context. In the terms of Mahowald et al. [4] LLMs have become amazingly proficient at *linguistic competence* — knowing *how* to say things; but they are not nearly as good at *functional competence* — knowing *what* to say. Specifically, they can be (relatively) easily fooled by, for example, asking for the result of a straightforward arithmetic problem that does not appear in their training corpus or by asking for the solution to a novel planning problem that requires knowledge about how the world works.

A Failure Example of GPT-4 in Planning

Problem (P1): You have 5 blocks. One cannot place more than one block on another block. b5 is on top of b3. b4 is on top of b2. b2 is on top of b1. b3 is on top of b4. b1 is on the table. b5 is clear. Your arm is empty.
Your goal is to move the blocks.
b1 should be on top of b2.
b3 should be on top of b5.
b4 should be on top of b1.

GPT-4:

```
1. Move b5 from b3 to the table.
2. Move b3 from b4 to b5.
3. Move b4 from b2 to the table.
4. Move b2 from b1 to b5.
5. Move b1 from the table to b2.
6. Move b4 from the table to b1.
```

Does that mean that we should increase efforts to include all arithmetic and planning problems in their training corpus? Clearly, that is a fool's errand. On the other hand, why should it be necessary? We already have calculators and general-purpose symbolic planners that are guaranteed to produce correct answers. Thus a natural alternative approach, and one that we are admittedly not the first to explore, is to connect LLMs to such tools.

With this motivation in mind, the objective of the research reported in this paper is, for the first time, to enable LLMs to solve planning problems *correctly*. We aim to do so without altering the LLMs themselves, even with finetuning [5], [6]. Rather, we introduce a methodology, called LLM+P by which, when posed a natural language description of a planning problem, the LLM:

1) outputs a problem description suitable as input to a

*Equal contribution.
†Department of Computer Science, The University of Texas at Austin {bliu, lqiang, joydeep, pstone}@cs.utexas.edu, jiangyuqian@utexas.edu
‡Department of Computer Science, State University of New York at Binghamton {xzhan244,zhangs}@binghamton.edu
§Sony AI
[1]The code and results are publicly available at https://github.com/Cranial-XIX/llm-pddl.git

Main contribution

Translates a natural-language planning problem into PDDL, delegates search to a classical planner, and translates the correct or optimal plan back into natural language.

Intuition

Use each component where it is strongest: the LLM handles flexible language, while a sound planning algorithm handles combinatorial search and validity.

Significance

Demonstrates a concrete neuro-symbolic reliability pattern and separates a fluent plan description from the external procedure that supplies correctness guarantees.

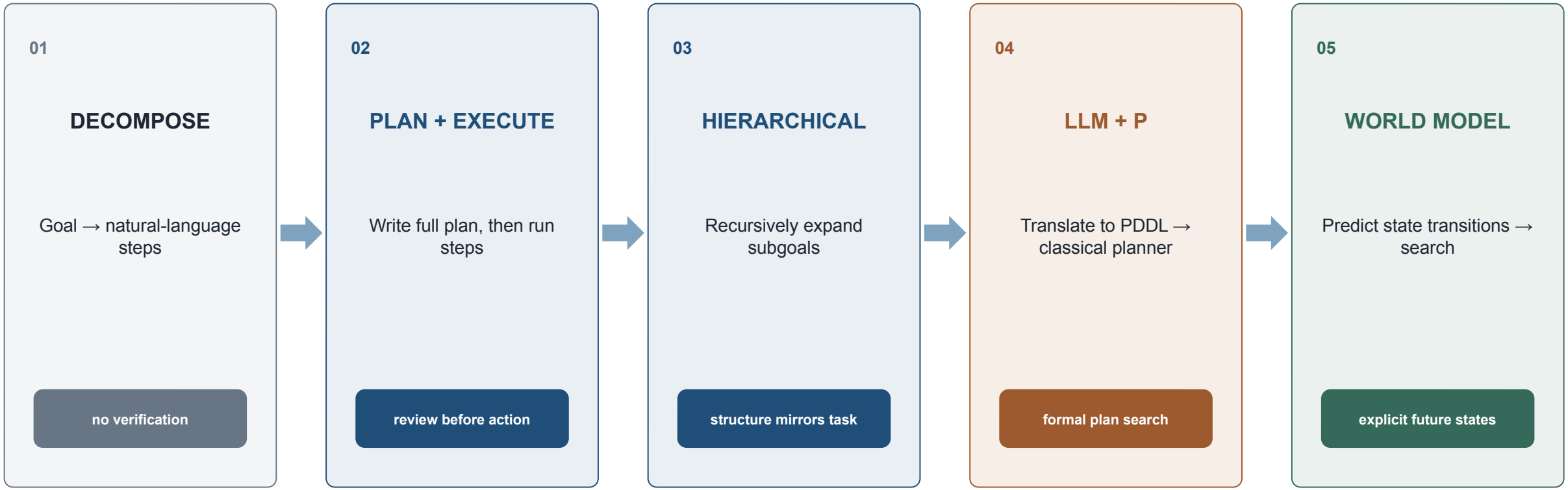
Open paper ↗

First page shown at left

Primary source: https://arxiv.org/abs/2304.11477

Planning architectures

More external structure generally means more reviewability and stronger guarantees



LLM contribution narrows as external verification becomes stronger →

Source: Week 2 notes §5.1

Paper spotlight: LLM-Modulo

LLMs Can't Plan, But Can Help Planning in LLM-Modulo Frameworks • ICML 2024

Position: LLMs Can't Plan, But Can Help Planning in LLM-Modulo Frameworks

Subbarao Kambhampati [1] Karthik Valmeekam [1] Lin Guan [1] Mudit Verma [1] Kaya Stechly [1] Siddhant Bhambri [1] Lucas Saldyt [1] Anil Murthy [1]

Abstract

We argue that auto-regressive LLMs cannot, by themselves, do planning or self-verification (which is after all a form of reasoning), and shed some light on the reasons for misunderstandings in the literature. We also argue that LLMs should be viewed as universal approximate knowledge sources that have much more meaningful roles to play in planning/reasoning tasks beyond simple front-end/back-end format translators. We present a vision of **LLM-Modulo Frameworks** that combines the strengths of LLMs with external model-based verifiers in a tighter bi-directional interaction regime. We will show how the models driving the external verifiers themselves can be acquired with the help of LLMs. We will also argue that rather than simply pipelining LLMs and symbolic components, this LLM-Modulo Framework provides a better *neuro-symbolic* approach that offers tighter integration between LLMs and symbolic components, extending the scope of model-based planning/reasoning regimes towards more flexible knowledge, problem and preference specifications.

1. Introduction

Large Language Models (LLMs), essentially n-gram models on steroids which have been pre-trained on web-scale language corpora (or, effectively, our collective consciousness), have caught the imagination of the AI research community with linguistic capabilities that no one expected text completion systems to possess. Their seeming versatility has led many researchers to wonder whether they can also do well on planning and reasoning tasks typically associated with System 2 competency. On the face of it, this doesn't seem to ring true, as both by training and operation, LLMs are best seen as a giant pseudo System 1 (Kahneman, 2011) (see Figure 1). Even from a pure engineering perspective, a system that takes constant time to produce the next token cannot possibly be doing principled reasoning on its own.[1] Not surprisingly, initial excitement based on anecdotal performance of LLMs on reasoning tasks (Bubeck et al., 2023) has been dissipated to some extent by the recent spate of studies, including our own, questioning the robustness of such behaviors–be they planning (Valmeekam et al., 2023c; Kambhampati, 2024), simple arithmetic and logic (Dziri et al., 2023), theory of mind (Ullman, 2023; Verma et al., 2024b), or general mathematical and abstract benchmarks (McCoy et al., 2023; Gendron et al., 2023). Despite this, a steady stream of claims continue to be made in the literature about the planning and reasoning capabilities of LLMs. In light of questions about their planning capabilities, the head-long rush into agentic LLMs should be particularly concerning. After all, acting without the ability to plan is surely a recipe for unpleasant consequences!

In an ironic juxtaposition to this unwarranted optimism about the planning and reasoning abilities of LLMs, there is also unwarranted pessimism about the roles LLMs can play in planning/reasoning tasks. Several efforts (e.g. (Liu et al., 2023; Pan et al., 2023; Xie et al., 2023)) advocate using LLMs only as glorified translators–converting reasoning problems embedded in textual format to symbolic representations, and pawning them off to external classical symbolic solvers (with all their attendant expressivity and search complexity challenges (Doyle & Patil, 1991)).[2]

In truth, LLMs can be a whole lot more than machine trans-

[1]School of Computing and AI, Arizona State University, Tempe, AZ, USA. Correspondence to: Subbarao Kambhampati <rao@asu.edu>.

Proceedings of the 41st International Conference on Machine Learning, Vienna, Austria. PMLR 235, 2024.

[1]Think of asking an LLM an yes/no question–*is this theorem logically entailed by this first-order logic knowledge-base*. This is well-known to be a semi-decidable problem. Ask yourself if the LLM will take longer in answering the question. (If you are thinking Chain-of-thought prompts or training with step-by-step data, consider that you are essentially changing the nature of the original prompt/training).

[2]In some circles, this unidirectional pipeline has been given the undeserved badge of *neuro-symbolic architecture*.

Main contribution

Argues against both unaided LLM planning and translator-only pessimism, proposing tighter bidirectional interaction between LLM knowledge sources and external model-based verifiers.

Intuition

Let an approximate generator propose knowledge and candidate plans while sound critics identify violations; feed those critiques back until a candidate satisfies the formal constraints.

Significance

Reframes reliable planning as modular cooperation and supplies a general architecture for combining flexible language models with verifiable reasoning systems.

Open paper ↗

First page shown at left

Primary source: https://arxiv.org/abs/2402.01817

LLMs can help planning-without being the planner

LLM-Modulo separates proposal from sound verification

LLM role

Idea generation • translation • repair suggestions

Verifier role

Classical planner • simulator • formal critic

Reliability comes from the critic-not from the plan's fluency.

Source: Kambhampati et al., LLM-Modulo (ICML 2024) • Week 2 notes §5.2

Live demo: same task, three conditions

Log correctness, tokens, calls, and wall-clock time

A

Plain CoT

single sample

B

ReAct

calculator / search

C

Tree-of-Thoughts

b = 3 • depth = 3

1. Which condition was correct?
2. Which failure mode was visible in the trace?
3. Was the accuracy gain worth the added cost?
4. Would a simpler method + cheap verifier be better?

Source: Week 2 notes §6, in-class demo

70-80′

10-minute break

Return with one sentence:

“The extra reasoning cost was / was not justified because…”

04

Read, present, and critique

Three passes, five talk parts, six critique lenses-and one worked ReAct review.

Paper spotlight: Three-pass reading

How to Read a Paper • ACM SIGCOMM CCR 2007

How to Read a Paper

S. Keshav
David R. Cheriton School of Computer Science, University of Waterloo
Waterloo, ON, Canada
keshav@uwaterloo.ca

ABSTRACT

Researchers spend a great deal of time reading research papers. However, this skill is rarely taught, leading to much wasted effort. This article outlines a practical and efficient *three-pass method* for reading research papers. I also describe how to use this method to do a literature survey.

Categories and Subject Descriptors: A.1 [Introductory and Survey]

General Terms: Documentation.

Keywords: Paper, Reading, Hints.

1. INTRODUCTION

Researchers must read papers for several reasons: to review them for a conference or a class, to keep current in their field, or for a literature survey of a new field. A typical researcher will likely spend hundreds of hours every year reading papers.

Learning to efficiently read a paper is a critical but rarely taught skill. Beginning graduate students, therefore, must learn on their own using trial and error. Students waste much effort in the process and are frequently driven to frustration.

For many years I have used a simple approach to efficiently read papers. This paper describes the 'three-pass' approach and its use in doing a literature survey.

2. THE THREE-PASS APPROACH

The key idea is that you should read the paper in up to three passes, instead of starting at the beginning and plowing your way to the end. Each pass accomplishes specific goals and builds upon the previous pass: The *first* pass gives you a general idea about the paper. The *second* pass lets you grasp the paper's content, but not its details. The *third* pass helps you understand the paper in depth.

2.1 The first pass

The first pass is a quick scan to get a bird's-eye view of the paper. You can also decide whether you need to do any more passes. This pass should take about five to ten minutes and consists of the following steps:

1. Carefully read the title, abstract, and introduction
2. Read the section and sub-section headings, but ignore everything else
3. Read the conclusions
4. Glance over the references, mentally ticking off the ones you've already read

At the end of the first pass, you should be able to answer the *five Cs*:

1. *Category*: What type of paper is this? A measurement paper? An analysis of an existing system? A description of a research prototype?
2. *Context*: Which other papers is it related to? Which theoretical bases were used to analyze the problem?
3. *Correctness*: Do the assumptions appear to be valid?
4. *Contributions*: What are the paper's main contributions?
5. *Clarity*: Is the paper well written?

Using this information, you may choose not to read further. This could be because the paper doesn't interest you, or you don't know enough about the area to understand the paper, or that the authors make invalid assumptions. The first pass is adequate for papers that aren't in your research area, but may someday prove relevant.

Incidentally, when you write a paper, you can expect most reviewers (and readers) to make only one pass over it. Take care to choose coherent section and sub-section titles and to write concise and comprehensive abstracts. If a reviewer cannot understand the gist after one pass, the paper will likely be rejected; if a reader cannot understand the highlights of the paper after five minutes, the paper will likely never be read.

2.2 The second pass

In the second pass, read the paper with greater care, but ignore details such as proofs. It helps to jot down the key points, or to make comments in the margins, as you read.

1. Look carefully at the figures, diagrams and other illustrations in the paper. Pay special attention to graphs. Are the axes properly labeled? Are results shown with error bars, so that conclusions are statistically significant? Common mistakes like these will separate rushed, shoddy work from the truly excellent.
2. Remember to mark relevant unread references for further reading (this is a good way to learn more about the background of the paper).

Main contribution

Provides a structured three-pass method that moves from overview, to argument and evidence, to detailed reconstruction or reproduction when the reader's purpose requires it.

Intuition

Do not spend maximum effort on every paper. Increase attention progressively, and stop when the current pass has answered the question you came to the paper with.

Significance

Makes literature reading scalable and gives students observable outputs for each pass instead of treating 'read the paper' as one vague, linear activity.

Open paper ↗

First page shown at left

Primary source: http://ccr.sigcomm.org/online/files/p83-keshavA.pdf

The three-pass method

Spend depth where it changes your understanding

PASS 1

5-10 min

SHAPE

Title • abstract • headings • figures • conclusion

State the problem + one-sentence contribution

PASS 2

30-60 min

ARGUMENT

Read in order • skip derivations • inspect evidence

Name baseline, metric, and varied knobs

PASS 3

core papers

RECONSTRUCT

Re-derive or run the released code

Find the gap between claim and implementation

Course expectation: Pass 1 on all readings • Pass 2 on your topic • Pass 3 on the core paper.

Source: Week 1 notes §3.1 • adapted from Keshav's three-pass approach • Course website §04

PRESENTATION STRUCTURE

Every seminar talk has five parts

Mechanism and critique are the spine; demo and open questions make claims checkable

1

Context + problem

Why this matters; what question is answered

2

Main approaches

Explain mechanisms, not labels

3

Critical analysis

Assumptions, evidence, comparisons

25% of grade

4

Demo / example

Show one worked trace or real run

5

Projects + open directions

End with a specific, falsifiable question

Source: Week 1 notes §3.2 • Course website §04 • seminar rubric

Paper spotlight: Agent evaluation

AI Agents That Matter • TMLR 2025

AI Agents That Matter

Sayash Kapoor*, Benedikt Stroebl*, Zachary S. Siegel, Nitya Nadgir, Arvind Narayanan

Princeton University
July 2, 2024

Abstract

AI agents are an exciting new research direction, and agent development is driven by benchmarks. Our analysis of current agent benchmarks and evaluation practices reveals several shortcomings that hinder their usefulness in real-world applications. First, there is a narrow focus on accuracy without attention to other metrics. As a result, SOTA agents are needlessly complex and costly, and the community has reached mistaken conclusions about the sources of accuracy gains. Our focus on cost in addition to accuracy motivates the new goal of jointly optimizing the two metrics. We design and implement one such optimization, showing its potential to greatly reduce cost while maintaining accuracy. Second, the benchmarking needs of model and downstream developers have been conflated, making it hard to identify which agent would be best suited for a particular application. Third, many agent benchmarks have inadequate holdout sets, and sometimes none at all. This has led to agents that are fragile because they take shortcuts and overfit to the benchmark in various ways. We prescribe a principled framework for avoiding overfitting. Finally, there is a lack of standardization in evaluation practices, leading to a pervasive lack of reproducibility. We hope that the steps we introduce for addressing these shortcomings will spur the development of agents that are useful in the real world and not just accurate on benchmarks.

1 Introduction

Compound AI systems, or AI agents, are becoming an important research direction. Zaharia et al. [63] argue that "compound AI systems will likely be the best way to maximize AI results in the future, and might be one of the most impactful trends in AI in 2024." Over a dozen agent benchmarks have been released, spanning domains such as web interaction [66], programming [21] and tool use [39]. Many benchmarks developed for LLM evaluation have also been used for agent evaluation.

Agent evaluation differs from language model evaluation in fundamental ways. Agents can be used on tasks that are harder, more realistic, have more real-world utility, and usually don't have a single correct answer. For example, agents can use the command line to carry out tasks; SWE-Agent even includes its own agent-computer interface [59]. Agents can cost much more than a single model call. For example, the authors of SWE-Agent capped *each* run of the agent at $4 USD, which translates to hundreds of thousands of language model tokens.

As a result, agent benchmarking comes with distinct challenges. This paper empirically demonstrates these challenges and provides recommendations for addressing them. Specifically, we make five contributions.

1. **AI agent evaluations must be cost-controlled (Section 2).** The language models underlying most AI agents are stochastic. This means simply calling the underlying model multiple times can increase accuracy [27, 6, 26]. We introduce three new simple baseline agents and empirically

*Equal Contribution. Contact: {sayashk,stroebl}@princeton.edu

Main contribution

Identifies recurring benchmark failures: accuracy-only reporting, ignored cost, conflated developer needs, weak holdouts, benchmark overfitting, and inconsistent reproducibility practices.

Intuition

An agent that wins a leaderboard by spending far more calls or exploiting benchmark quirks may be less useful than a simpler system at the same cost and under a clean holdout.

Significance

Provides the methodological foundation for jointly examining task success, cost, robustness, generalization, and reproducibility in both seminar critiques and course projects.

Open paper ↗

First page shown at left

Primary source: https://arxiv.org/abs/2407.01502

Six lenses turn summary into analysis

A paper's claim is only as strong as its weakest untested assumption

ASSUMPTIONS

What must be true-but is not tested?

BASELINES

Is the comparison fair at matched compute/cost?

METRICS

Does the number measure the actual claim?

GENERALIZATION

Different model, task, domain, or scale?

REPRODUCIBILITY

Seeds, variance, artifacts, contamination?

CROSS-PAPER

Do methods or conclusions conflict?

Habit: "This paper's result would not hold if…"

Source: Week 1 notes §3.3 • Course website §04

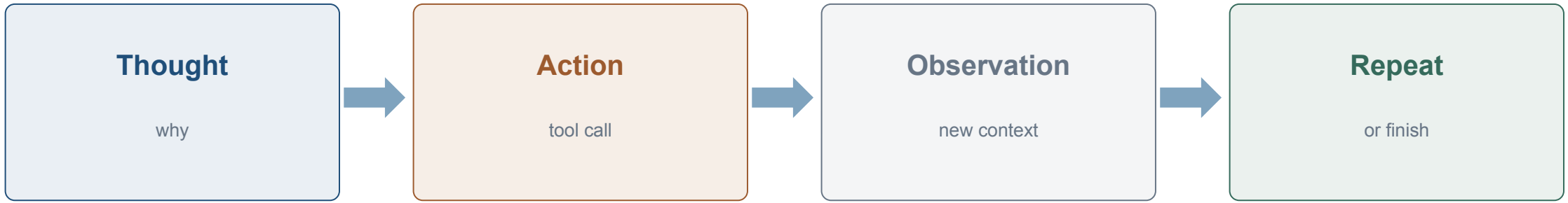
Worked critique: ReAct

First reconstruct the claim and mechanism before judging the evidence

CLAIM

Interleaving reasoning traces with actions and observations improves task success over CoT-only or acting-only baselines on knowledge-intensive QA and interactive decision tasks.

MECHANISM



Before critique: can you redraw this loop from memory and say what the thought actually does?

Source: Yao et al., ReAct (ICLR 2023) • Week 1 notes §3.4 • Course website §05

ReAct under four checks

Separate supported structure from open assumptions and dated empirical scope

ASSUMPTION

Actions are cheap, fast, reversible; observations are benign.

Untested for paid, slow, destructive, or adversarial tools.

BASELINE

CoT-only and Act-only are roughly matched in few-shot setup.

Missing retrieve-then-answer at matched token budget.

GENERALIZATION

Evidence comes from HotpotQA, FEVER, ALFWorld, WebShop.

Open: does the gain persist with stronger current models?

GAIN SOURCE

Both reasoning-only and acting-only underperform the combination.

Supports a genuine structural-not only compute-claim.

Verdict: foundational loop; dated numbers; action-cost assumption should be tested, not accepted.

Source: Week 1 notes §3.4 • Course website §05

Activity: turn an abstract into a falsifiable claim

Scope the claim, then map evidence and the missing experiment

ORIGINAL

“Our method enables reliable autonomous planning.”

SCOPED

“On [task/domain], under [conditions], method X improves [metric] over [baseline] at [budget].”

1. Which experiment supports each bracket?
2. Which bracket is missing or ambiguous?
3. What result would falsify the scoped claim?
4. Which critique lens matters most?

Source: Week 1 discussion question 4 • Course website §06

05

Organization & next actions

Groups, seminar expectations, project direction, and the Week 2 reading pack.

How the course runs

One group, two seminars, one research-style project

1 Groups

3-4 students

Form or swap groups and rank seminar topics before this lecture ends.

Same group across both seminars and the project.

Avoid consecutive presentation weeks where possible.

2 Seminars

30-minute talk + 15-minute defense

Slides due 48 hours before.

Everyone submits reading notes and two critical questions.

Peer critique is graded.

3 Project

Agent/MAS architecture with tools/MCP + memory

Quantitative evaluation and security analysis required.

Proposal → checkpoint → report, code, demo, defense.

AI use is allowed where stated-declare it, verify it, and be able to defend it.

Source: PLAN.md §§4-8

Before Week 2

Tools, MCP, context, evaluation-and proposal review

01	**Form/swap group + rank seminar topics**	Complete in class • before 20:29
02	**Submit project idea 1-pager**	Sun 13 Sep • 23:59 ICT
03	**Submit D1 proposal + prepare 5′ pitch**	Sun 13 Sep • 23:59 • 3-4 pages
04	**Read three core sources**	Before class Wed 16 Sep • 18:00

Course site → Week 1 quiz + code exercise

Lecture notes → Week 2: Reasoning & Planning

Source: PLAN.md §4 • Week 1 website §08

TAKEAWAYS

If you remember nothing else

Five claims to carry into every seminar and project decision

1 **LOOP** — An agent perceives, decides, acts, observes, and repeats.

2 **SCOPE** — Name which component-and autonomy level-a paper actually changes.

3 **COST** — More reasoning calls are useful only when the added evidence justifies them.

4 **FEEDBACK** — Reflection and search are bounded by evaluator quality.

5 **CRITIQUE** — Ask what would make the result stop holding.

Return to your weakest objective: what changed?

Source: Week 1 website takeaway • Week 1-2 lecture notes

READ NEXT

References & course resources

Core sources used in this deck

Foundations

- Sumers et al. - CoALA (TMLR 2024)
- Wei et al. - Chain-of-Thought (NeurIPS 2022)
- Wang et al. - Self-Consistency (ICLR 2023)
- Yao et al. - ReAct (ICLR 2023)
- Shinn et al. - Reflexion (NeurIPS 2023)
- Yao et al. - Tree of Thoughts (NeurIPS 2023)
- Besta et al. - Graph of Thoughts (AAAI 2024)
- Zhou et al. - LATS (ICML 2024)

Methodology & planning

- Keshav - How to Read a Paper
- Kambhampati et al. - LLM-Modulo (ICML 2024)
- Huang et al. - Cannot Self-Correct Yet (ICLR 2024)
- Kapoor et al. - AI Agents That Matter (TMLR 2025)
- Anthropic - Building Effective Agents (2024)

Course materials

- Lecture Notes/week-01…md
- Lecture Notes/week-02…md
- CO5151-Website/dist/week-01.html

Interactive Week 1 site

GitHub repository

Questions?